Databases para agentes: PostgreSQL y pgvector

Máster AI Engineer · Sesión 9 · Ignacio de Pastors

thePower Tech School

AGENDA

01 Bloque teórico
02 Bloque práctico
03 Ejercicio para casa
04 Cierre y dudas



thePower
Tech School

01

Bloque teórico

thePower
Tech School

Por qué un agente necesita una base de datos vectorial

Memoria persistente
El modelo olvida entre conversaciones. Guardamos fuera lo relevante: soporte que recuerda incidencias previas.

Búsqueda por significado
No por palabra exacta: escribes “no me carga la app” y encuentra “error de inicio de sesión”.

Motor del RAG
Lo que montaremos en la sesión 10/9 se apoya justo en esto: recuperar contexto y responder con tus datos.

ANALOGÍA
Un buscador normal es un Ctrl+F: buscas “coche” y, si el documento dice “automóvil”, no lo encuentra. Un buscador vectorial sí: entiende que “coche”, “automóvil” y “vehículo” hablan de lo mismo, y devuelve el documento aunque no comparta ni una palabra.
Buscamos ideas, no palabras.

thePower Tech School

Embeddings: convertir significado en números

Textos parecidos = vectores cercanos
El vector captura el significado: 'médico' y 'doctor' casi coinciden.

Distancia coseno
Mide el ángulo entre vectores: 0 = significado idéntico.

sentence-transformers, local y gratis
paraphrase-multilingual-MiniLM-L12-v2 (all-MiniLM-L6-v2 si todo es en inglés), 384 dimensiones, sin pagar API.

Caso práctico
Busca en 5.000 FAQs aunque el usuario use otras palabras.

EJEMPLO
'perro' -> [0.21, -0.4, 0.8, ...]
'gato' -> [0.19, -0.3, 0.7, ...]
'ordenador' -> [-0.7, 0.5, -0.2, ...]



Cerca en el espacio = cerca en significado

thePower Tech School

PostgreSQL + pgvector: tu base de datos de siempre, ahora vectorial

Postgres + extensión pgvector
La BD de siempre, open source y gratis, aprende un truco nuevo.

Nuevo tipo de columna vector(384)
Guardas el embedding como una columna más de la tabla.

Todo en un solo sitio, con SQL
JOIN con clientes, pedidos o permisos en la misma query.

Caso real: buscador interno de documentación
La misma query filtra por departamento y ordena por similitud.

CÓDIGO

```
CREATE EXTENSION vector;

CREATE TABLE docs (
  texto text,
  departamento text,
  embedding vector(384));
```

Dos líneas de SQL y tu BD ya es vectorial

thePower Tech School

Búsqueda: exacta vs. índices aproximados

BÚSQUEDA EXACTA

- Recorre todos los vectores: escaneo secuencial
- Recall 100%: siempre da el mejor resultado
- Se nota a partir de cientos de miles de filas
- Perfecta en desarrollo y en tablas pequeñas
- **Catálogo de 2M de productos: tarda segundos**

ÍNDICE HNSW

- Grafo por capas: saltas de barrio en barrio hasta la calle exacta
- Milisegundos sobre millones de vectores
- Recall típico del 95-99%
- Se ajusta con los parámetros m y ef_search
- **Ese mismo catálogo de 2M: responde en milisegundos**

En clase, exacta. En producción, HNSW.

thePower Tech School

Cómo se busca: el operador de vecinos más cercanos

1. **La pregunta también se vectoriza**
   Mismo modelo que al indexar; si no, los vectores no son comparables.

2. **El operador <=> mide la distancia**
   Distancia coseno en pgvector; también hay <-> (L2) y <#> (producto interno).

3. **ORDER BY distancia + LIMIT k**
   Te quedas con los k trozos más relevantes, típicamente de 3 a 5.

Ejemplo
"¿Cómo cancelo mi suscripción?" recupera "baja del servicio".

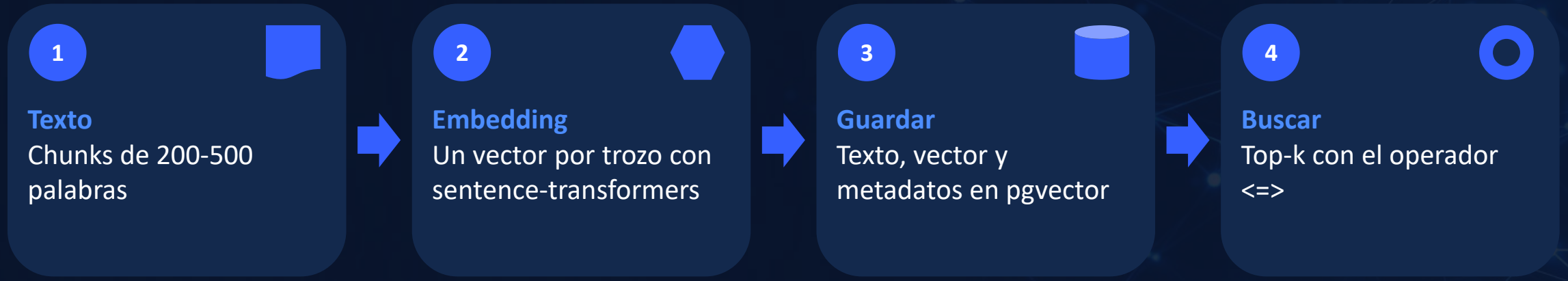
CÓDIGO

```
SELECT texto
FROM docs
ORDER BY embedding <=> :consulta
LIMIT 3;
```

Cuatro líneas de SQL = búsqueda semántica

thePower Tech School

El flujo completo: de un documento a una respuesta



Este flujo es exactamente el indexado del RAG que montaremos en la sesión 14

Ejemplo: 300 PDFs de políticas internas -> 4.000 chunks -> respuestas citando la fuente

thePower Tech School

Multi-tenancy: una base de datos, muchos clientes

El riesgo: fuga entre clientes
Una query sin filtro devuelve documentos de otra empresa; en un buscador vectorial el fallo es silencioso, no da error.

Solución 1: columna tenant_id
Etiquetas cada fila y filtras siempre; sencillo, pero depende de la disciplina del equipo.

Solución 2: Row-Level Security
La propia base de datos aplica el filtro, aunque se te olvide en la consulta.

Caso real: asistente de RR. HH.
El empleado de la empresa A jamás puede recuperar el convenio de la empresa B.

IDEA CLAVE

El aislamiento no es una feature: es un requisito

Aislar los datos de cada cliente sostiene la seguridad y la confianza en cualquier producto real.

Además suele ser una exigencia legal: el RGPD obliga a separar y proteger los datos de cada empresa.

thePower Tech School

Dos formas de aislar los datos por cliente

FILTRO POR tenant_id

- Cada fila lleva su cliente en la columna tenant_id
- Todas las queries añaden WHERE tenant_id = :cliente
- Simple y suficiente para aprender, pero depende de que el programador no se olvide nunca
- **Ejemplo: la misma consulta sin WHERE devuelve todo**

ROW-LEVEL SECURITY

- Se activa con ALTER TABLE ... ENABLE ROW LEVEL SECURITY y una policy
- Postgres aplica el filtro por debajo, en cada consulta
- Imposible saltárselo por descuido: lo recomendado en producción
- **Ejemplo: la misma consulta sin WHERE devuelve solo lo tuyo**

En clase, tenant_id. En producción, RLS.

thePower Tech School

02

Bloque práctico

thePower
Tech School

Punto de partida: una base de datos que entiende de significados

1

PostgreSQL + pgvector

Tres formas, mismo código: Docker (pgvector/pgvector), la nube gratis (Neon o Supabase) o un Postgres local con la extensión.

2

Embeddings en local

sentence-transformers con un modelo multilingüe de 384 dimensiones. Gratis y sin mandar datos a ninguna API.

3

Tres paquetes nuevos

psycopg (hablar con Postgres), pgvector (el tipo vector en Python) y sentence-transformers.

EL HILO DE LA HISTORIA

En la sesión 5 el bot se inventaba que enviamos a Portugal. Hoy guardamos las políticas REALES de la tienda en una base de datos que busca por significado. En la sesión 10, el RAG las usará para responder sin inventar.

Y de bonus: PostGIS, para que el mismo Postgres entienda también de mapas.

thePower Tech School

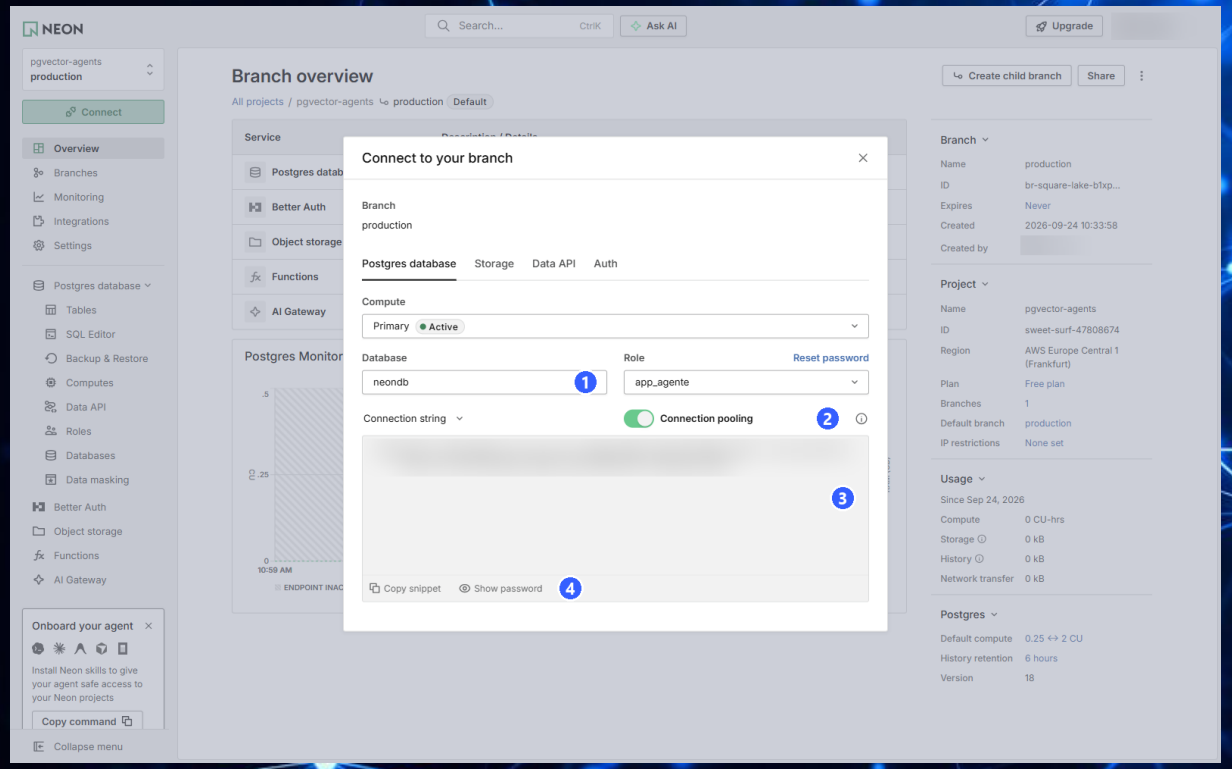
Antes de empezar: Neon en 3 minutos

NEON · console.neon.tech (plan gratuito)

1. Registrarse (Google, GitHub o email)
2. New project: nombre y región Europa (Frankfurt)
3. Servicios: solo Postgres database
4. Connect -> Show password -> copiar la connection string
5. Pegarla en el .env como DATABASE_URL

SIN DOCKER Y SIN INSTALAR NADA

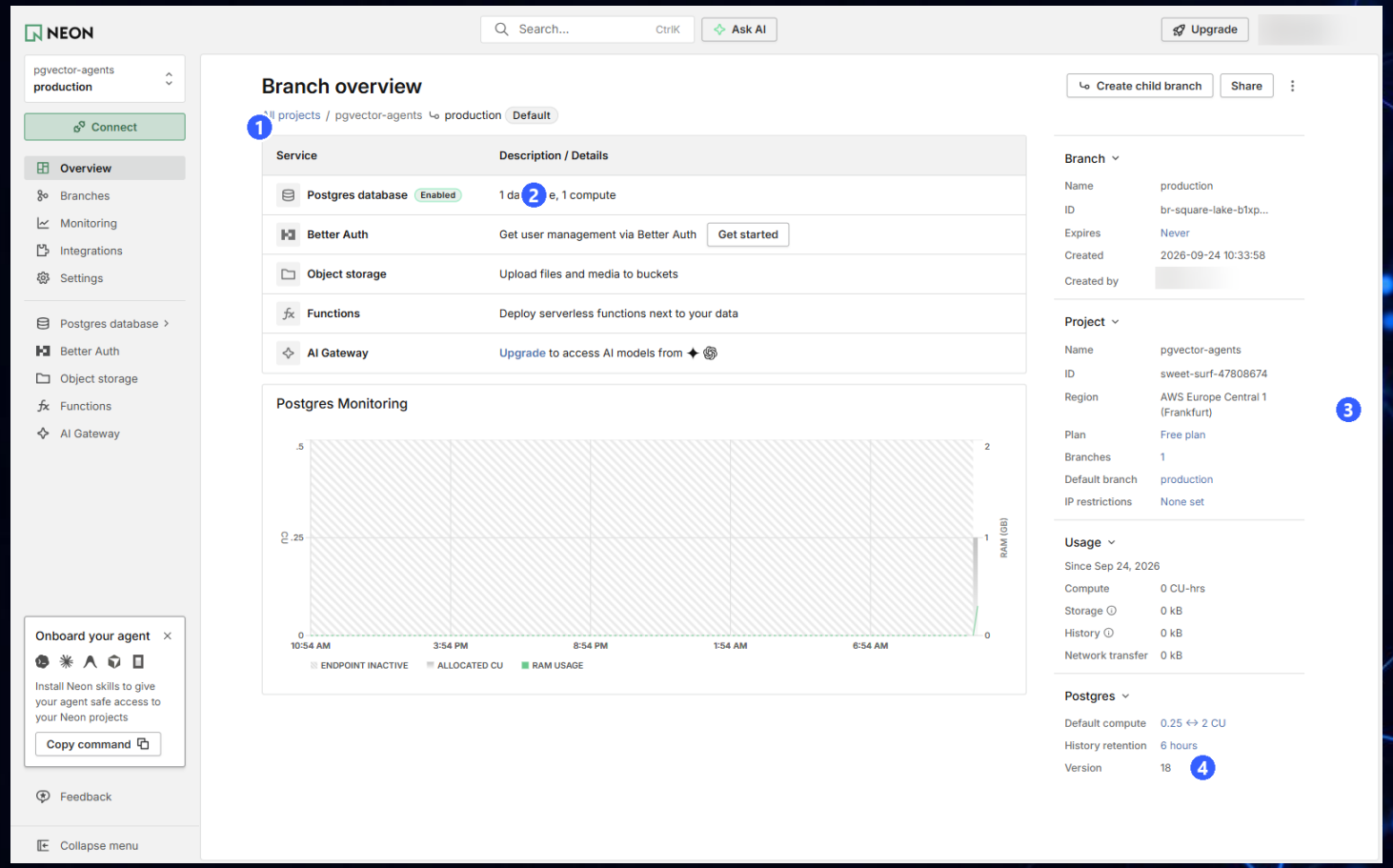
pgvector y PostGIS vienen incluidos. Ignorad el 'agent prompt' que ofrece Neon: solo necesitamos la cadena.



La cadena lleva la contraseña: va al .env, que nunca se comparte.

thePower Tech School

El panel de Neon: tu Postgres en la nube



1. **Connect**
   de aquí sale la cadena de conexión

2. **Postgres database**
   1 base de datos y 1 compute, activos

3. **Región y plan**
   Frankfurt, plan gratuito

4. **Versión**
   PostgreSQL 18

thePower Tech School

Paso 1 · Objetivo

1

OBJETIVO DEL PASO

Una base de datos que guarda vectores

DE LA TEORÍA

PostgreSQL + pgvector: tu base de datos de siempre, con una columna nueva de tipo vector.

LO QUE VAMOS A VER

- CREATE EXTENSION vector
- La tabla documentos
- Las extensiones en Neon

ARCHIVO

- 01_setup_bd.py

thePower Tech School

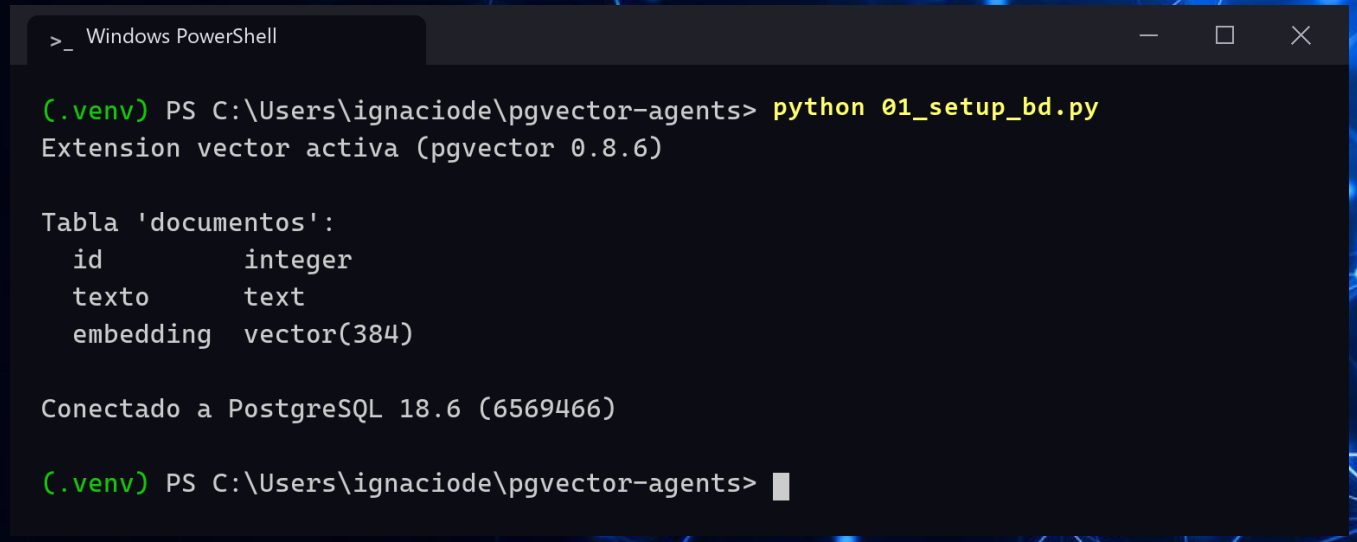
Paso 1 · Postgres + pgvector

QUÉ HACEMOS

- Conectar con la cadena del .env
- Activar pgvector: CREATE EXTENSION vector
- Tabla documentos con columna vector(384)
- Enseñar la tabla creada

CONCEPTO CLAVE

Postgres no necesita ser una base de datos 'exótica': con una extensión aprende el tipo vector y lo guarda como una columna más.



FÍJATE EN

pgvector 0.8.6 activo y una tabla normal cuya última columna es vector(384). PostgreSQL 18 en Neon.

thePower Tech School

Paso 1 · Anatomía: la extensión y la tabla

01_setup_bd.py

```python
conn = conectar(registrar_vector=False)   # la extension todavia no existe

# 1) Activar pgvector: a partir de aqui Postgres entiende el tipo "vector"
conn.execute("CREATE EXTENSION IF NOT EXISTS vector")
version = conn.execute("SELECT extversion FROM pg_extension WHERE extname = 'vector'").fetchone()[0]
print(f"Extension vector activa (pgvector {version})")

# 2) Una tabla normal de Postgres... con una columna de 384 numeros
conn.execute("DROP TABLE IF EXISTS documentos")
conn.execute(f"""
    CREATE TABLE documentos (
        id        SERIAL PRIMARY KEY,
        texto     TEXT NOT NULL,
        embedding VECTOR({DIMENSIONES})
    )
""")
```

1 **CREATE EXTENSION vector**
una vez por base de datos: Postgres aprende el tipo vector

2 **La tabla**
id y texto de siempre + embedding VECTOR(384)

thePower Tech School

Paso 1 · Anatomía: conexión y modelo en un solo sitio

db.py

```python
# Modelo local y gratuito: textos -> vectores de 384 dimensiones.
# Multilingue: all-MiniLM-L6-v2 (solo ingles) falla con frases en espanol (ver extras/paso3_el_modelo_importa.py)
MODELO = "paraphrase-multilingual-MiniLM-L12-v2"
DIMENSIONES = 384


def conectar(registrar_vector: bool = True) -> psycopg.Connection:
    """Abre la conexion con la cadena del .env (Neon, Supabase, Docker o local: el codigo es el mismo)."""
    conn = psycopg.connect(os.environ["DATABASE_URL"], autocommit=True)
    if registrar_vector:
        register_vector(conn)   # para pasar listas/arrays de Python como columnas vector
    return conn


_modelo = None


def embedding(textos):
    """Convierte uno o varios textos en embeddings (el modelo se descarga la primera vez, ~470 MB)."""
    global _modelo
    if _modelo is None:
        from sentence_transformers import SentenceTransformer
        _modelo = SentenceTransformer(MODELO)
    return _modelo.encode(textos, normalize_embeddings=True)
```

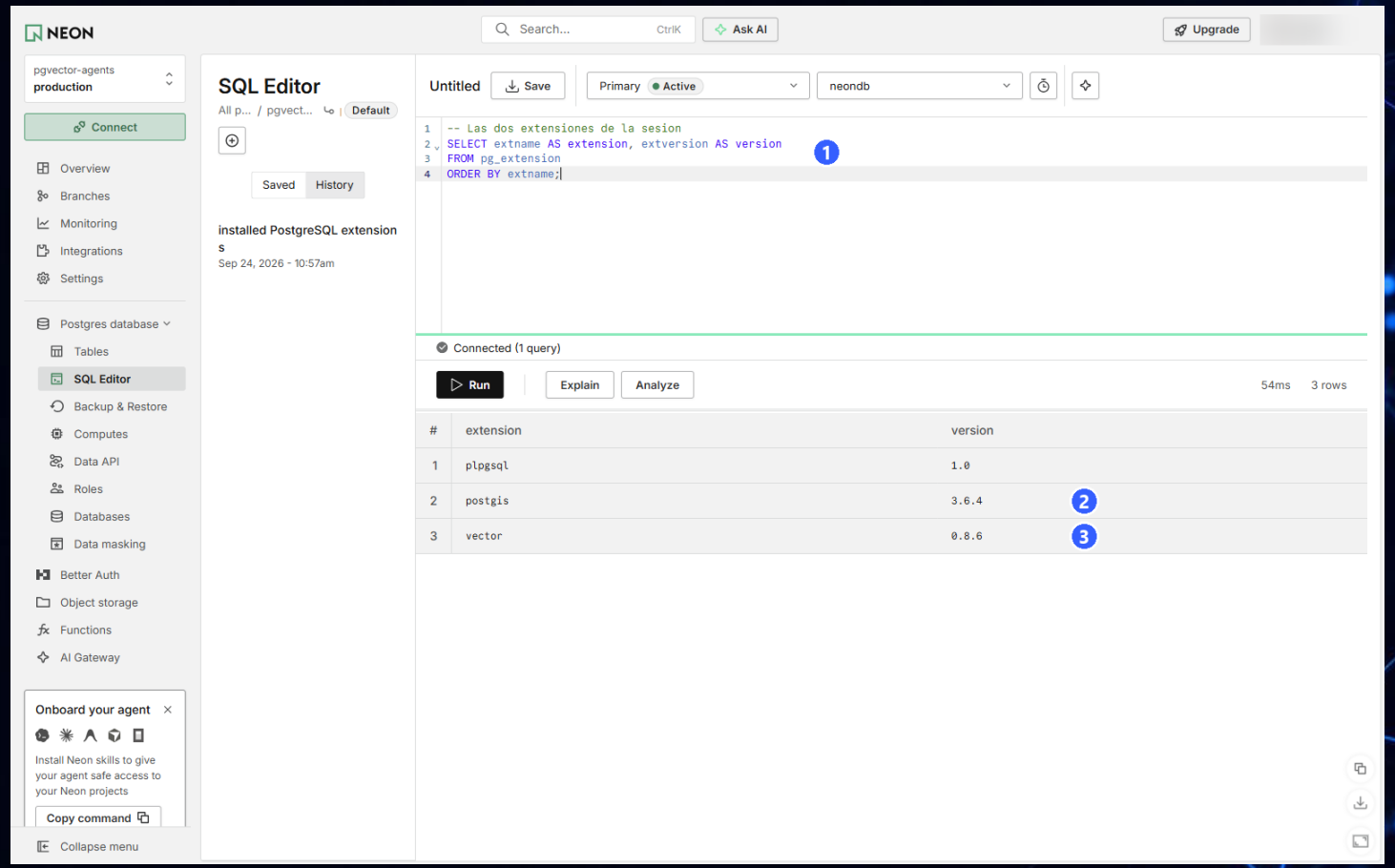
1. **El modelo**
   multilingüe, 384 dimensiones: debe coincidir con la columna

2. **conectar()**
   la cadena del .env: Neon, Docker o local, mismo código

3. **embedding()**
   textos -> vectores normalizados; se descarga la primera vez

thePower Tech School

Paso 1 · Las extensiones en Neon



1 **La consulta**
pg_extension: qué extensiones tiene la base de datos

2 **postgis 3.6.4**
la del bonus: mapas y distancias

3 **vector 0.8.6**
pgvector: la de hoy

thePower Tech School

Paso 1 · Pregunta a la clase

PREGUNTA A LA CLASE

Si ya hay bases de datos vectoriales como Pinecone o Chroma, ¿por qué usar Postgres?

Pensad en dónde están ya los datos de vuestra empresa.

thePower Tech School

Paso 1 · Lo que te llevas

- ✓ **Una extensión** CREATE EXTENSION vector convierte Postgres en base vectorial
- ✓ **Una columna más** embedding VECTOR(384) junto a tus datos de siempre
- ✓ **Mismo código en cualquier sitio** Neon, Supabase, Docker o local: cambia solo la cadena
- ✓ **Las dimensiones importan** 384 = las del modelo; si cambias de modelo, cambias la columna

SIGUIENTE Rellenar la tabla: convertir textos en vectores con un modelo local.

thePower Tech School

Paso 2 · Objetivo

2

OBJETIVO DEL PASO

Convertir textos en vectores y guardarlos

DE LA TEORÍA

Embeddings: un vector que captura el significado. Textos parecidos = vectores cercanos.

LO QUE VAMOS A VER

- 10 políticas de la tienda
- Su vector en la base de datos
- Qué es 'estar cerca'

ARCHIVO

- 02_embeddings.py

thePower Tech School

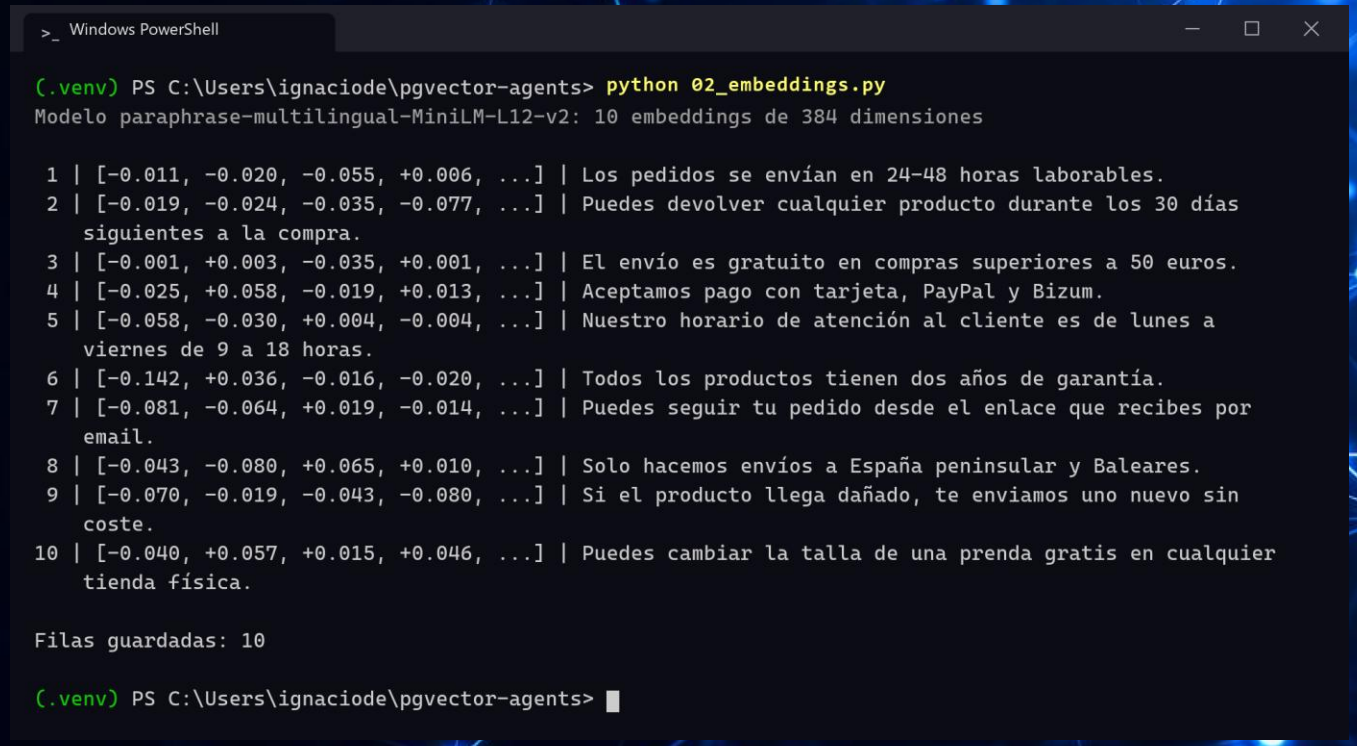
Paso 2 · Generar y guardar embeddings

QUÉ HACEMOS

- Modelo local multilingüe
- 10 políticas reales de la tienda
- Insertar texto + vector
- Ver lo guardado

CONCEPTO CLAVE

Cada texto se convierte en 384 números que representan su significado. Se guardan junto al texto, en la misma fila.



FÍJATE EN

cada política con el principio de su vector: números pequeños, positivos y negativos. Sin ellos no hay búsqueda.

thePower Tech School

Paso 2 · Anatomía: de texto a fila

02_embeddings.py

```python
conn = conectar()
vectores = embedding(CORPUS)   # 10 textos -> 10 vectores de 384 numeros (1)
print(f"Modelo {MODELO}: {len(vectores)} embeddings de {len(vectores[0])} dimensiones\n")

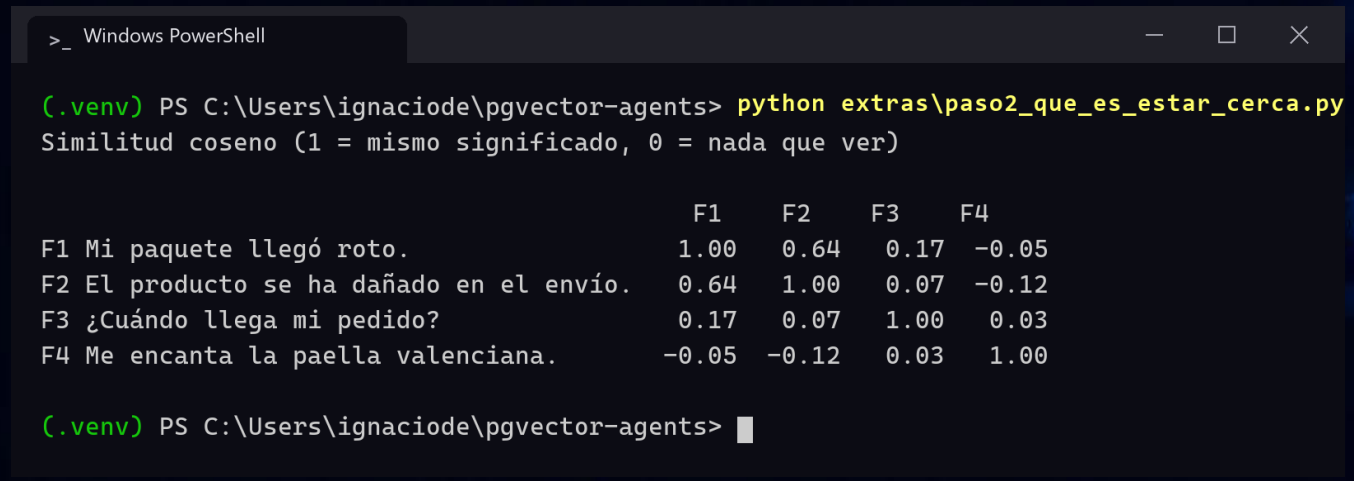
conn.execute("TRUNCATE documentos RESTART IDENTITY") (2)
with conn.cursor() as cur:
    cur.executemany("INSERT INTO documentos (texto, embedding) VALUES (%s, %s)", list(zip(CORPUS, vectores)))

# Comprobar lo guardado: el texto y el principio de su vector
for id_, texto, emb in conn.execute("SELECT id, texto, embedding FROM documentos ORDER BY id"): (3)
    inicio = ", ".join(f"{x:+.3f}" for x in emb.to_numpy()[:4])
    print(f"{id_:>2} | [{inicio}, ...] | {texto}")
```

1. **embedding(CORPUS)**
   10 textos -> 10 vectores de 384 números, de una vez
2. **executemany**
   texto y vector entran juntos en cada INSERT
3. **Comprobar**
   leer lo guardado: el vector vuelve como objeto Vector

thePower Tech School

Paso 2 · Variación: ¿qué es estar cerca?



VARIACIÓN EN VIVO

QUÉ CAMBIAMOS

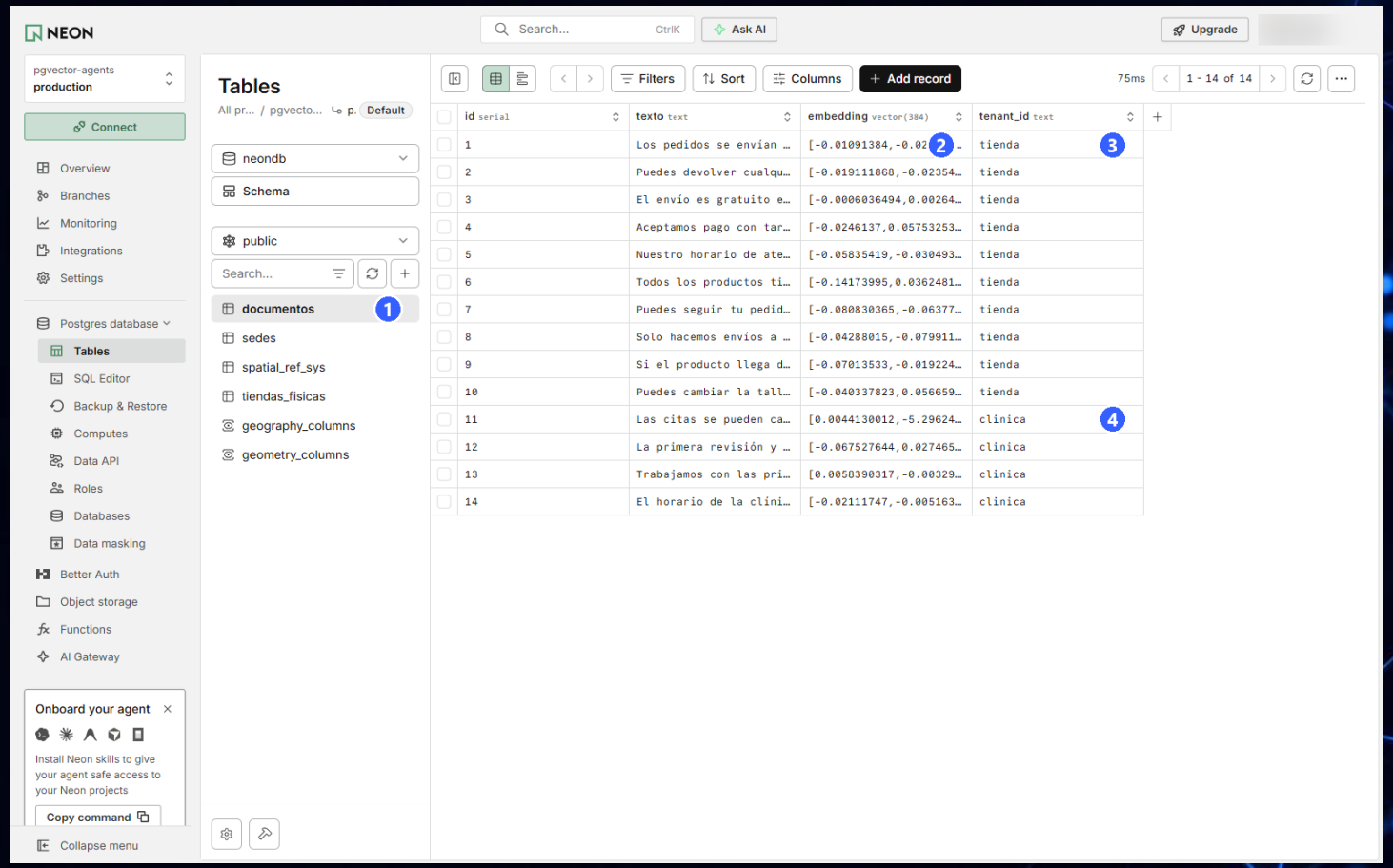
En vez de guardar, comparamos 4 frases entre sí y calculamos la similitud coseno de cada pareja.

QUÉ DEMUESTRA

Dos frases sobre un paquete dañado: 0,64, aunque no comparten palabras. La paella: casi 0 con todo. Eso es 'estar cerca'.

thePower Tech School

Paso 2 · La tabla en Neon



1. **documentos**
   la tabla, en Tables

2. **embedding vector(384)**
   la columna de vectores, como cualquier otra

3. **tenant_id**
   (la añadimos en el paso 4)

4. **clínica**
   (los documentos del segundo cliente, paso 4)

thePower Tech School

Paso 2 · Zoom: la columna embedding

id serial	texto text	embedding vector(384)	tenant_id text
1	Los pedidos se envían …	[-0.01091384,-0.020009…	tienda
2	Puedes devolver cualqu…	[-0.019111868,-0.02354…	tienda
3	El envío es gratuito e…	[-0.0006036494,0.00264…	tienda
4	Aceptamos pago con tar…	[-0.0246137,0.05753253…	tienda
5	Nuestro horario de ate…	[-0.05835419,-0.030493…	tienda
6	Todos los productos ti…	[-0.14173995,0.0362481…	tienda
7	Puedes seguir tu pedid…	[-0.080830365,-0.06377…	tienda
8	Solo hacemos envíos a …	[-0.04288015,-0.079911…	tienda
9	Si el producto llega d…	[-0.07013533,-0.019224…	tienda
10	Puedes cambiar la tall…	[-0.040337823,0.056659…	tienda
11	Las citas se pueden ca…	[0.0044130012,-5.29624…	clinica
12	La primera revisión y …	[-0.067527644,0.027465…	clinica
13	Trabajamos con las pri…	[0.0058390317,-0.00329…	clinica
14	El horario de la clíni…	[-0.02111747,-0.005163…	clinica

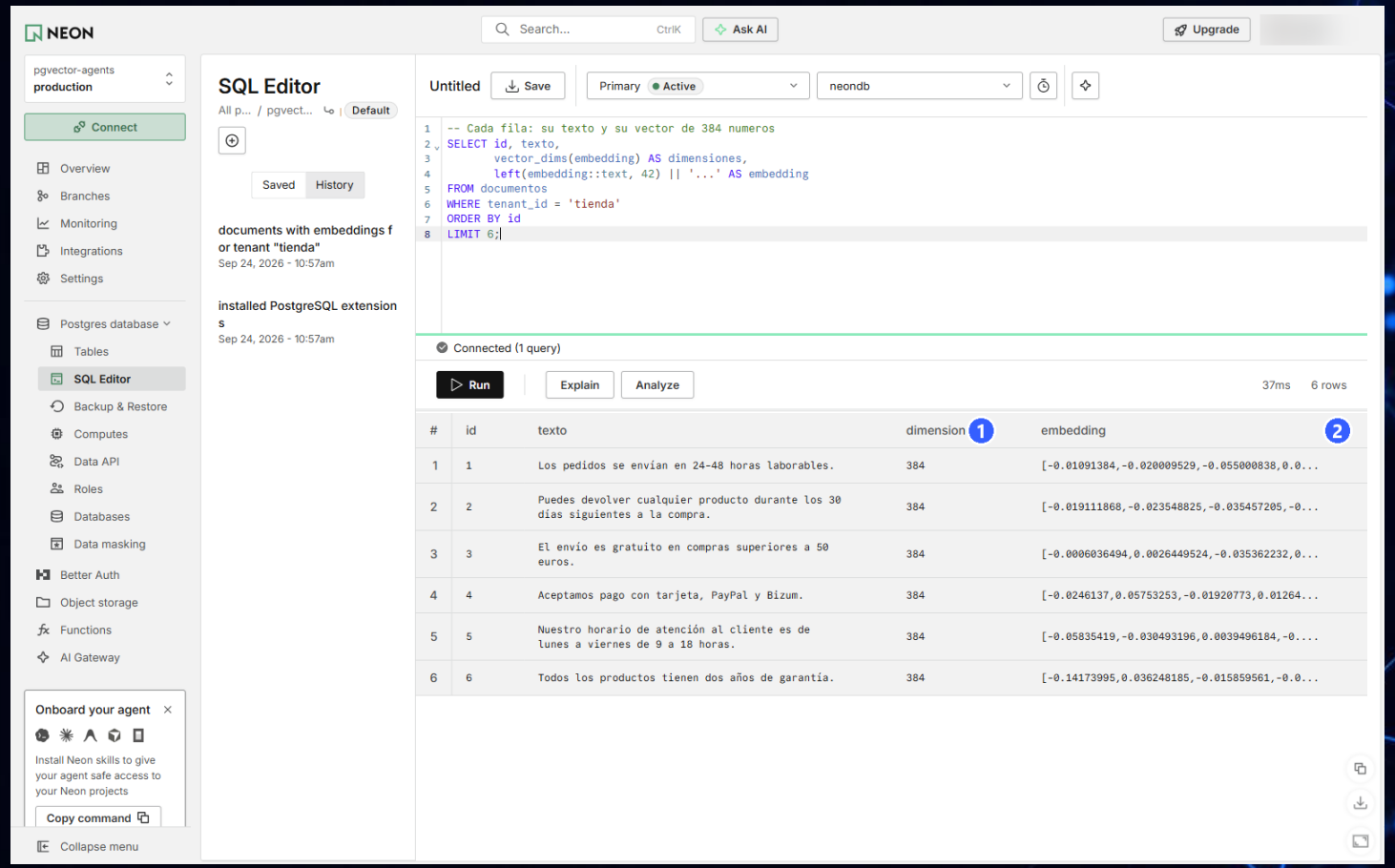
NEON · TABLES

Así guarda Postgres el significado

- Una fila por política: id, texto, embedding
- El vector se ve como texto: [-0.0109, -0.0200, ...]
- Son 384 números por fila: aquí solo se ven los primeros
- Para la base de datos es un dato más, como una fecha o un precio

thePower Tech School

Paso 2 · Consultar los vectores con SQL



1 **vector_dims**
384 en todas las filas: la dimensión del modelo

2 **embedding::text**
el principio de cada vector, legible

thePower Tech School

Paso 2 · Pregunta a la clase

PREGUNTA A LA CLASE

Dos frases no comparten ninguna palabra y aun así tienen similitud 0,64. ¿Cómo es posible?

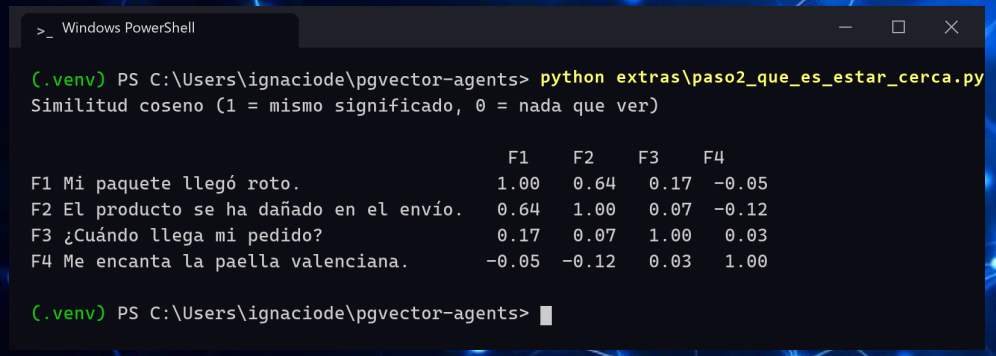
Recordad la matriz: 'paquete llegó roto' y 'producto dañado en el envío'.

thePower Tech School

Paso 2 · La respuesta

¿Cómo es posible?

- El modelo no compara palabras: compara significados
- Se ha entrenado con millones de frases y ha aprendido que 'roto' y 'dañado' se usan en contextos parecidos
- Por eso sus vectores apuntan en direcciones parecidas
- Y por eso 'paella' queda lejos de todo: habla de otra cosa



thePower Tech School

Paso 2 · Lo que te llevas

- ✓ **Embeddings en local** sentence-transformers: gratis y sin enviar datos fuera
- ✓ **Texto + vector** en la misma fila, con un INSERT normal
- ✓ **Similitud coseno** 1 = mismo significado, 0 = nada que ver
- ✓ **Todo visible en Neon** Tables y SQL Editor muestran los vectores

SIGUIENTE Buscar: la pregunta también se convierte en vector.

thePower Tech School

Paso 3 · Objetivo

3

OBJETIVO DEL PASO

Buscar por significado, no por palabras

DE LA TEORÍA

El operador <=> de pgvector: distancia coseno para encontrar los vecinos más cercanos.

LO QUE VAMOS A VER

- Top-3 para una pregunta
- Palabras frente a significado
- Que el modelo importa

ARCHIVO

- 03_busqueda.py

thePower Tech School

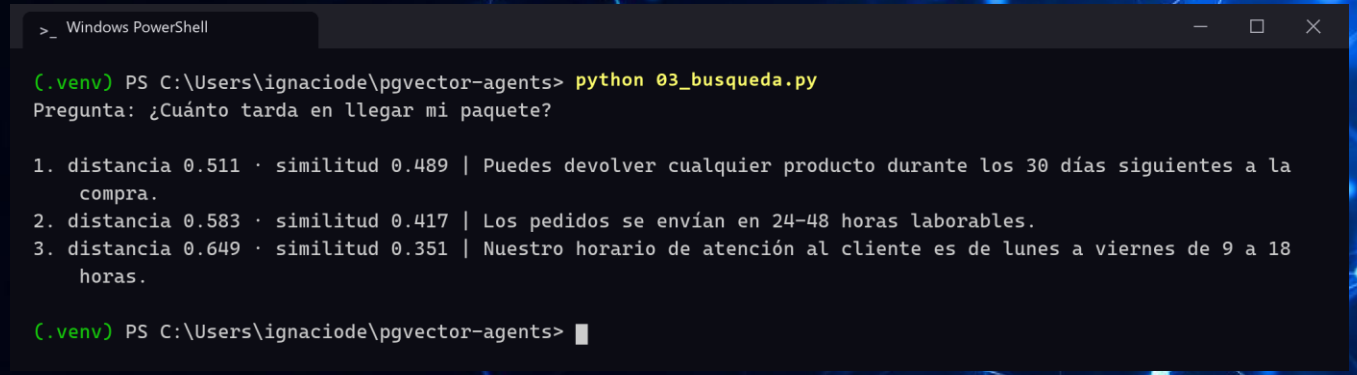
Paso 3 · Búsqueda por similitud

QUÉ HACEMOS

- La pregunta se convierte en vector
- ORDER BY embedding <=> pregunta
- LIMIT 3: los más parecidos
- Distancia y similitud de cada uno

CONCEPTO CLAVE

Buscar es ordenar por distancia: la pregunta se vectoriza con el MISMO modelo y Postgres devuelve los vecinos más cercanos.



FÍJATE EN

la buena ('se envían en 24-48 horas') sale SEGUNDA. El top-3 existe para esto: la primera no siempre es la mejor.

thePower Tech School

Paso 3 · Anatomía: la consulta de vecinos

03_busqueda.py

```python
q = embedding(pregunta)   # la pregunta se convierte en vector con el MISMO modelo  (1)

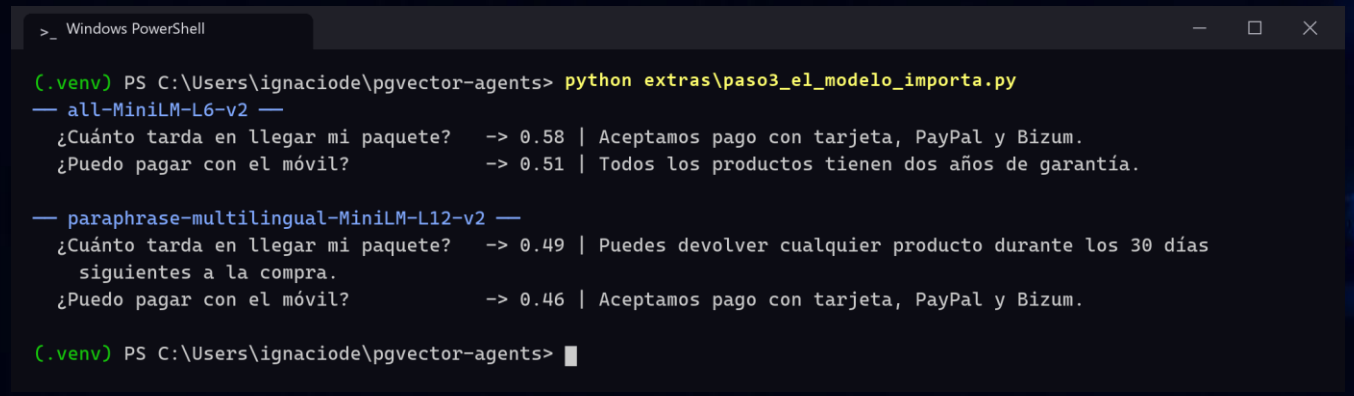
# <=> es la distancia coseno: 0 = mismo significado, 2 = opuesto. Ordenar por ella = vecinos mas cercanos  (2)
resultados = conn.execute("""
    SELECT texto, embedding <=> %s AS distancia
    FROM documentos
    ORDER BY embedding <=> %s
    LIMIT 3
""", (q, q)).fetchall()
```

1. **La pregunta, vectorizada**
   con el MISMO modelo que los documentos

2. **<=> y ORDER BY**
   distancia coseno: ordenar = vecinos más cercanos; LIMIT 3

thePower Tech School

Paso 3 · Variación: el modelo importa



VARIACIÓN EN VIVO

QUÉ CAMBIAMOS

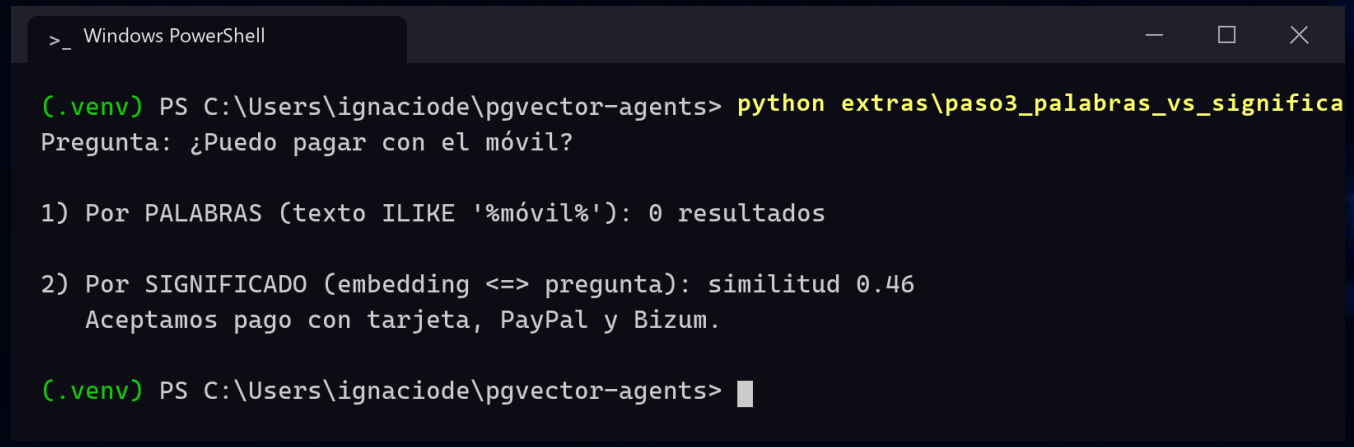
Mismo corpus y mismas preguntas en español con dos modelos: all-MiniLM-L6-v2 (inglés) y el multilingüe.

QUÉ DEMUESTRA

Con el modelo en inglés, '¿puedo pagar con el móvil?' devuelve la garantía. Con el multilingüe, Bizum. Mismo código, otro modelo, otro resultado.

thePower Tech School

Paso 3 · Variación: palabras frente a significado



VARIACIÓN EN VIVO

QUÉ CAMBIAMOS

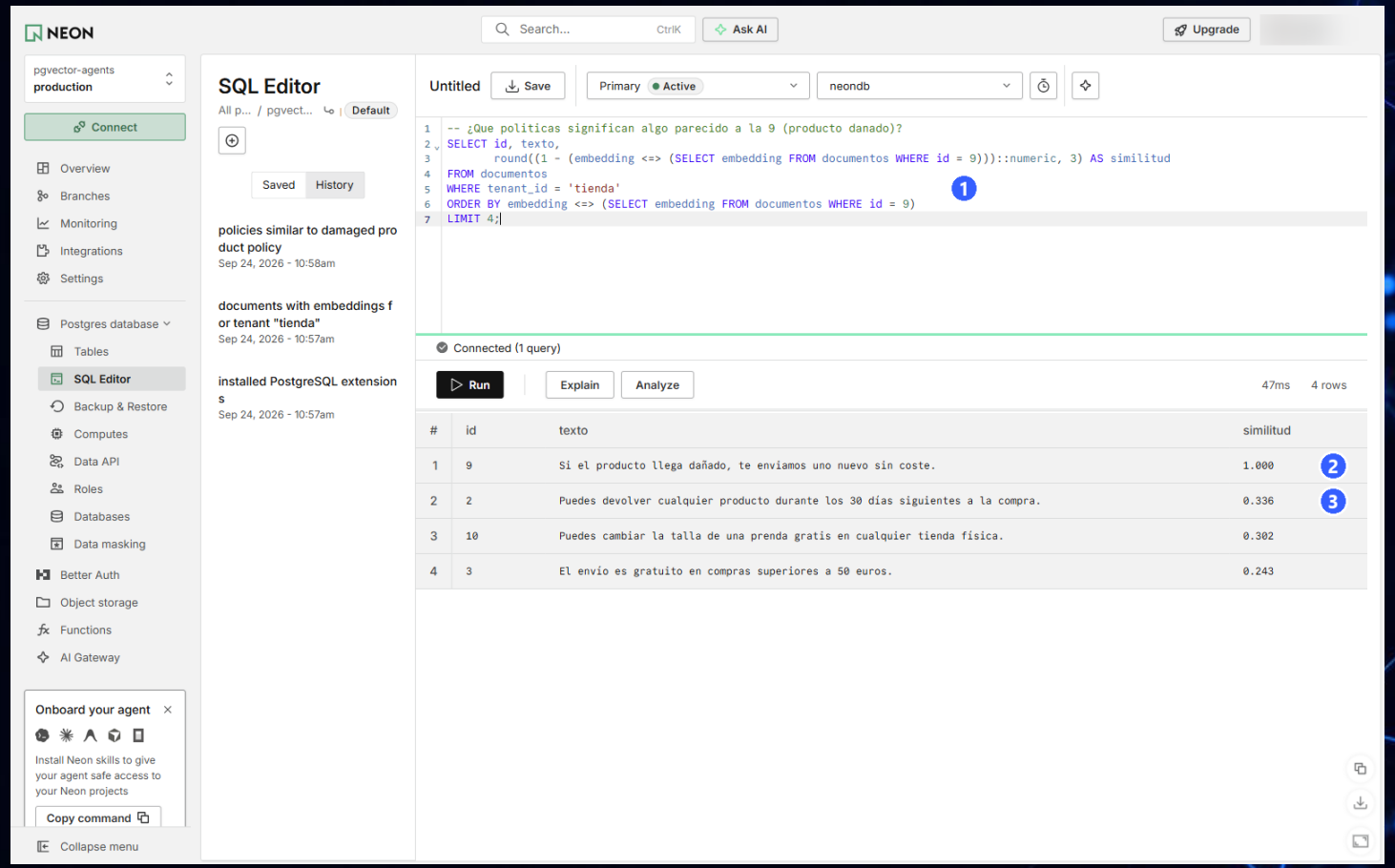
La misma pregunta buscada dos veces: con texto ILIKE '%movil%' y con el operador <=>.

QUÉ DEMUESTRA

Por palabras: 0 resultados, porque ninguna política dice 'móvil'. Por significado: 'Aceptamos Bizum'. Es la razón de ser de pgvector.

thePower Tech School

Paso 3 · Vecinos por significado en Neon



1. **Subconsulta**
   el vector de la política 9 como consulta, sin Python

2. **Ella misma**
   similitud 1,000: distancia 0

3. **Su vecina**
   devoluciones: la más parecida a 'producto dañado'

thePower Tech School

Paso 3 · Zoom: los vecinos de 'producto dañado'

#	id	texto	similitud
1	9	Si el producto llega dañado, te enviamos uno nuevo sin coste.	1.000
2	2	Puedes devolver cualquier producto durante los 30 días siguientes a la compra.	0.336
3	10	Puedes cambiar la talla de una prenda gratis en cualquier tienda física.	0.302
4	3	El envío es gratuito en compras superiores a 50 euros.	0.243

NEON · SQL EDITOR

Lo que la base de datos considera parecido

- 9 · dañado -> nuevo sin coste: 1,000
- 2 · devoluciones en 30 días: 0,336
- 10 · cambio de talla: 0,302
- 3 · envío gratis: 0,243 (ya poco que ver)

thePower Tech School

Paso 3 · Pregunta a la clase

PREGUNTA A LA CLASE

La respuesta correcta salió segunda. ¿Cómo lo mejoraríais?

Pensad en el modelo, en los textos y en cuántos resultados traemos.

thePower Tech School

Paso 3 · La respuesta

¿Cómo mejorar la búsqueda?

- Traer varios (top-k) y dejar que el LLM elija: es lo que hará el RAG en la sesión 10
- Un modelo mejor para el idioma: ya vimos la diferencia entre inglés y multilingüe
- Mejores textos: frases más completas se vectorizan mejor
- Y en la sesión 11, un reranker que reordena los resultados

thePower Tech School

Paso 3 · Lo que te llevas

- ✓ **<=>** distancia coseno; ORDER BY + LIMIT = vecinos más cercanos
- ✓ **Mismo modelo** la pregunta se vectoriza igual que los documentos
- ✓ **Significado, no palabras** ILIKE no encuentra lo que <=> sí
- ✓ **El modelo importa** en español, un modelo multilingüe

SIGUIENTE Escalar y separar clientes: índice HNSW y multi-tenancy.

thePower Tech School

Paso 4 · Objetivo

4

OBJETIVO DEL PASO

Rápido a escala y cada cliente con lo suyo

DE LA TEORÍA

Búsqueda exacta frente a índices aproximados (HNSW) y multi-tenancy: tenant_id y Row-Level Security.

LO QUE VAMOS A VER

- Índice HNSW
- Dos clientes, una tabla
- RLS: aislamiento en la base de datos

ARCHIVO

- 04_hnsw_tenants.py

thePower Tech School

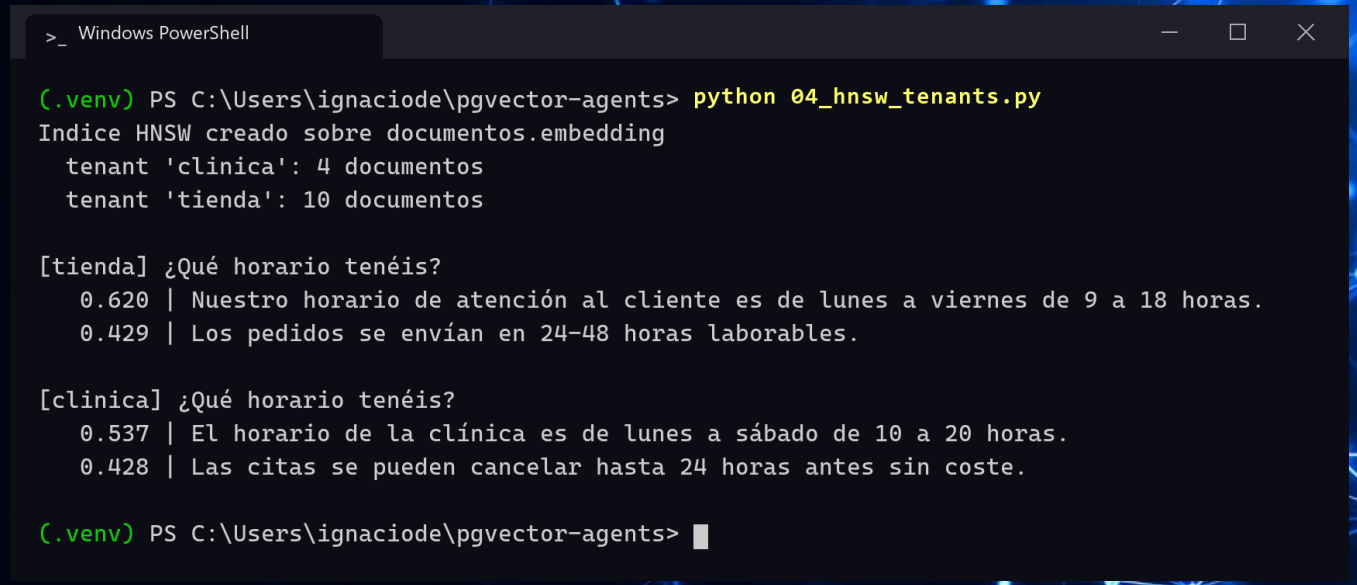
Paso 4 · Índice HNSW + multi-tenancy

QUÉ HACEMOS

- Índice HNSW sobre embedding
- Columna tenant_id
- Segundo cliente: una clínica dental
- La misma pregunta, filtrada por cliente

CONCEPTO CLAVE

Una base de datos, muchos clientes: cada búsqueda lleva SIEMPRE el filtro del cliente que pregunta.



FÍJATE EN

la misma pregunta, '¿qué horario tenéis?', devuelve el horario de la tienda o el de la clínica según quién pregunte.

thePower Tech School

Paso 4 · Anatomía: índice y aislamiento

04_hnsw_tenants.py

```python
# 1) Indice HNSW: busqueda aproximada muy rapida sobre la distancia coseno
conn.execute("CREATE INDEX IF NOT EXISTS documentos_embedding_hnsw ON documentos USING hnsw (embedding vector_cosine_ops)")
print("Indice HNSW creado sobre documentos.embedding")

# 2) Multi-tenancy: una columna tenant_id. Los documentos actuales son de la tienda...
conn.execute("ALTER TABLE documentos ADD COLUMN IF NOT EXISTS tenant_id TEXT NOT NULL DEFAULT 'tienda'")

# ...y damos de alta un segundo cliente: una clinica dental
conn.execute("DELETE FROM documentos WHERE tenant_id = 'clinica'")
with conn.cursor() as cur:
    cur.executemany("INSERT INTO documentos (texto, embedding, tenant_id) VALUES (%s, %s, 'clinica')",
                    list(zip(CLINICA, embedding(CLINICA))))
for tenant, n in conn.execute("SELECT tenant_id, count(*) FROM documentos GROUP BY tenant_id ORDER BY tenant_id"):
    print(f"  tenant '{tenant}': {n} documentos")


# 3) La misma busqueda, SIEMPRE filtrada por el cliente que pregunta
def buscar(pregunta: str, tenant: str, k: int = 2):
    q = embedding(pregunta)
    return conn.execute("""
        SELECT texto, embedding <=> %s AS distancia
        FROM documentos
        WHERE tenant_id = %s                -- el filtro que aisla a cada cliente
        ORDER BY embedding <=> %s
        LIMIT %s
    """, (q, tenant, q, k)).fetchall()
```

1. **HNSW**
   USING hnsw (embedding vector_cosine_ops)
2. **tenant_id**
   una columna con el cliente de cada fila
3. **Segundo cliente**
   la clínica, en la misma tabla
4. **buscar()**
   WHERE tenant_id = %s en TODAS las consultas

thePower Tech School

Paso 4 · Una tabla, dos clientes

#	tenant_id	documentos	ejemplo
1	clinica	4	El horario de la clínica es de lunes a sábado de 10 a 20 horas.
2	tienda	10	Aceptamos pago con tarjeta, PayPal y Bizum.

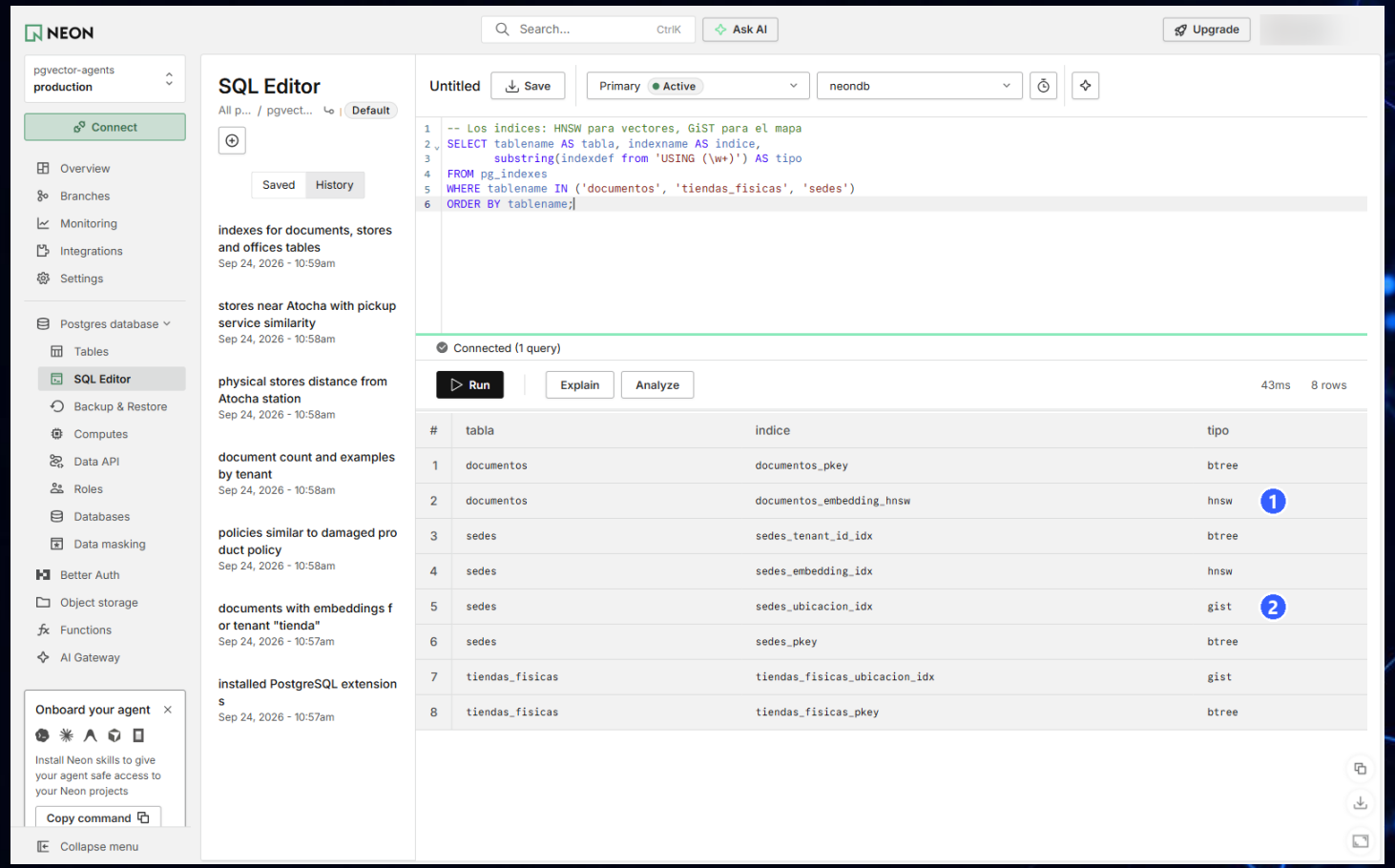
NEON · SQL EDITOR

Contado en Neon

- tienda: 10 documentos
- clínica: 4 documentos
- Misma tabla, mismo índice, datos separados por tenant_id
- El riesgo: una consulta sin el WHERE mezcla clientes

thePower Tech School

Paso 4 · Los índices en Neon

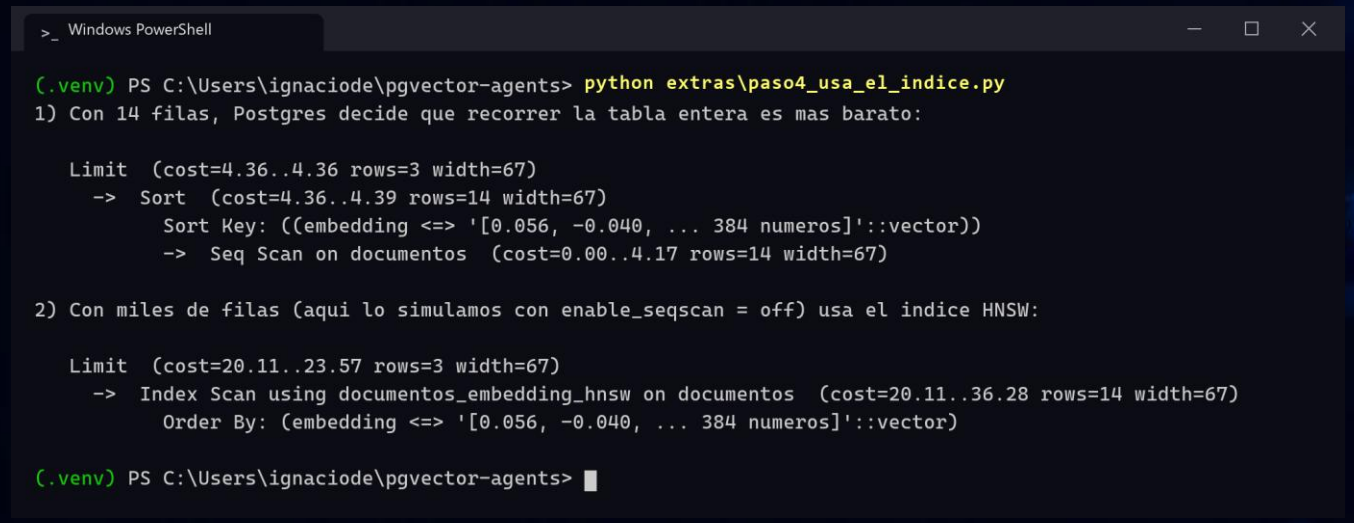


1. **hnsw**
   índices vectoriales en documentos y sedes

2. **gist**
   índices espaciales para el mapa (bonus)

thePower Tech School

Paso 4 · Variación: ¿usa Postgres el índice?



VARIACIÓN EN VIVO

QUÉ CAMBIAMOS

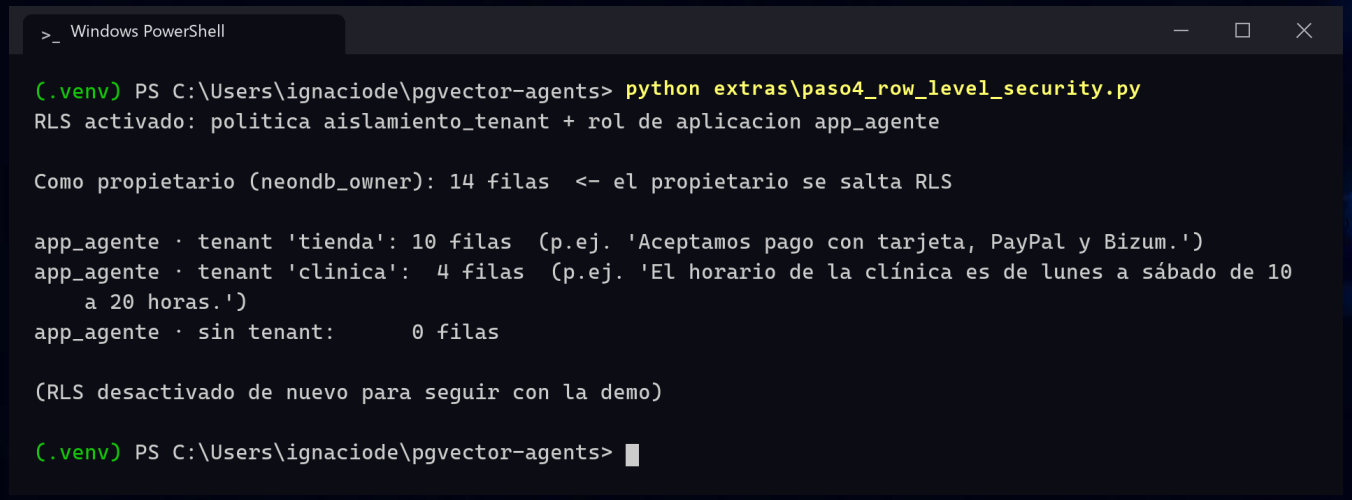
Pedimos a Postgres su plan con EXPLAIN: primero tal cual y después simulando una tabla grande.

QUÉ DEMUESTRA

Con 14 filas recorre la tabla entera (Seq Scan): es más barato. Cuando el volumen lo justifica, usa el HNSW (Index Scan). El índice es para escalar.

thePower Tech School

Paso 4 · Variación: Row-Level Security



VARIACIÓN EN VIVO

QUÉ CAMBIAMOS

La base de datos aísla sola: política por app.tenant_id y un rol de aplicación sin privilegios.

QUÉ DEMUESTRA

El propietario ve las 14 filas: se salta RLS. El rol de la app ve 10 o 4 según el cliente, y 0 sin cliente, con la misma consulta sin WHERE.

thePower Tech School

Paso 4 · Anatomía: RLS paso a paso

extras/paso4_row_level_security.py

```python
# 1) Activar RLS y la politica: solo se ven las filas del tenant de la sesion
conn.execute("ALTER TABLE documentos ENABLE ROW LEVEL SECURITY")
conn.execute("DROP POLICY IF EXISTS aislamiento_tenant ON documentos")
conn.execute("""CREATE POLICY aislamiento_tenant ON documentos
                USING (tenant_id = current_setting('app.tenant_id', true))""")

# 2) Un rol de aplicacion sin privilegios, con permiso solo de lectura
if not conn.execute("SELECT 1 FROM pg_roles WHERE rolname = 'app_agente'").fetchone():
    conn.execute("CREATE ROLE app_agente NOLOGIN")
conn.execute("GRANT app_agente TO CURRENT_USER WITH SET TRUE")
conn.execute("GRANT SELECT ON documentos TO app_agente")
print("RLS activado: politica aislamiento_tenant + rol de aplicacion app_agente\n")

usuario = conn.execute("SELECT current_user").fetchone()[0]
n = conn.execute(CONSULTA).fetchone()[0]
print(f"Como propietario ({usuario}): {n} filas  <- el propietario se salta RLS\n")

# 3) La MISMA consulta, sin WHERE, como app_agente y con dos clientes distintos
conn.execute("SET ROLE app_agente")
for tenant in ("tienda", "clinica"):
    conn.execute("SELECT set_config('app.tenant_id', %s, false)", (tenant,))
    n, ejemplo = conn.execute(CONSULTA).fetchone()
    print(f"app_agente · tenant '{tenant}': {n:>2} filas  (p.ej. '{ejemplo}')")
conn.execute("SELECT set_config('app.tenant_id', '', false)")
print(f"app_agente · sin tenant:     {conn.execute(CONSULTA).fetchone()[0]:>2} filas")
conn.execute("RESET ROLE")
```

1. **La política**
   tenant_id = current_setting('app.tenant_id')
2. **El rol de la app**
   sin privilegios y solo lectura
3. **La sesión**
   SET ROLE + set_config: la consulta ya no lleva WHERE

thePower Tech School

Paso 4 · Pregunta a la clase

PREGUNTA A LA CLASE

Con RLS activado, el propietario seguía viendo las 14 filas. ¿Por qué es peligroso en producción?

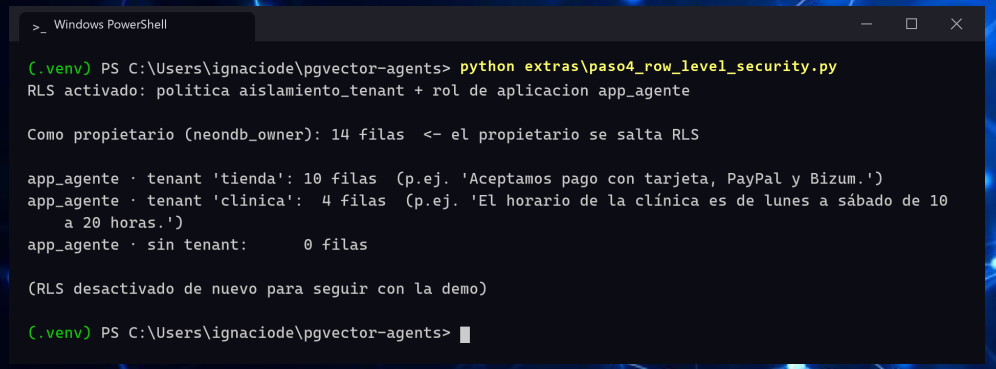
¿Con qué usuario se conecta vuestra aplicación?

thePower Tech School

Paso 4 · La respuesta

¿Por qué es peligroso?

- Si la app se conecta como propietario, RLS no protege nada
- La cadena que copiamos de Neon es la del propietario: cómoda para clase, mala para producción
- En producción: un rol de aplicación sin privilegios, con permisos mínimos
- RLS + tenant_id en el código: dos capas mejor que una



thePower Tech School

Paso 4 · Lo que te llevas

- ✓ **HNSW** índice aproximado: se nota a partir de miles de filas
- ✓ **EXPLAIN** para ver si Postgres lo usa
- ✓ **tenant_id** el filtro en cada consulta aísla a cada cliente
- ✓ **RLS** aislamiento en la base de datos, con un rol sin privilegios

SIGUIENTE Bonus: el mismo Postgres también entiende de mapas.

thePower Tech School

Bonus · Objetivo

5

OBJETIVO DEL PASO

Bonus GIS: la tienda más cercana que ofrece lo que buscas

DE LA TEORÍA

Postgres + PostGIS: tipos geográficos, distancias reales e índices espaciales, junto a los vectores.

LO QUE VAMOS A VER

- PostGIS en Neon
- Distancia frente a significado
- El mapa de las tiendas

ARCHIVO

- 05_bonus_gis.py

thePower Tech School

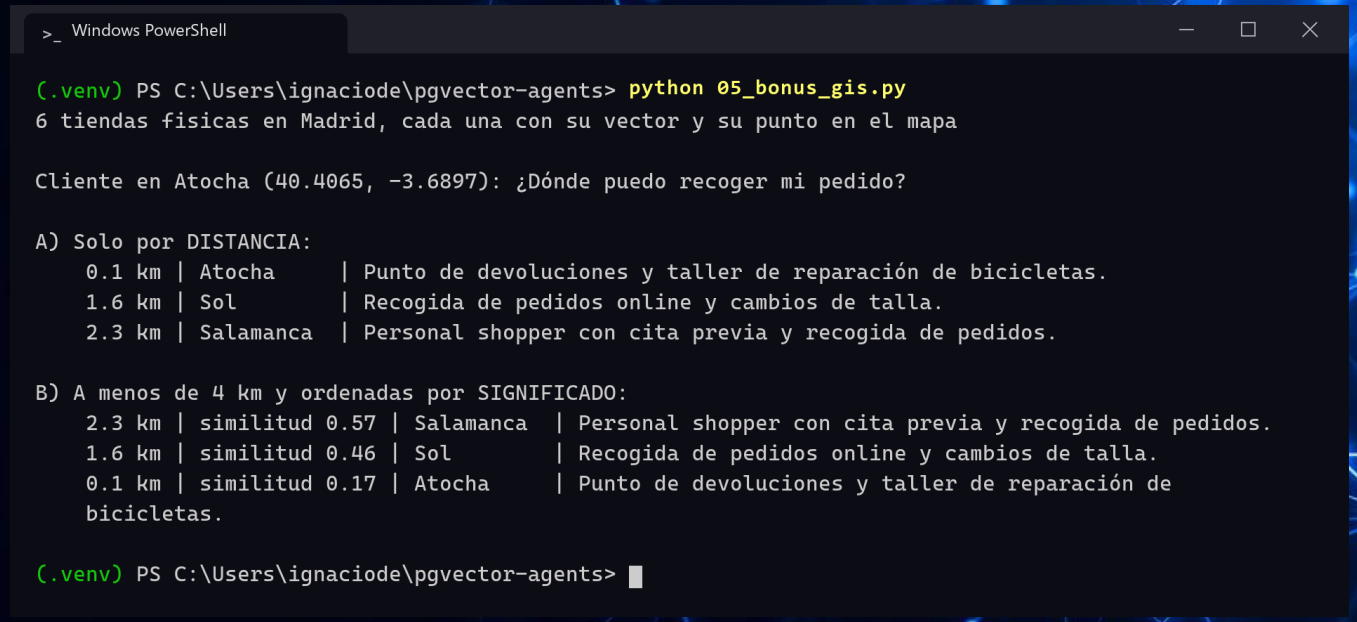
Bonus · PostGIS + pgvector

QUÉ HACEMOS

- Activar PostGIS
- 6 tiendas físicas en Madrid
- Cliente en Atocha: '¿dónde recojo mi pedido?'
- Solo distancia frente a distancia + significado

CONCEPTO CLAVE

Una sola consulta SQL que filtra por distancia real (PostGIS) y ordena por significado (pgvector).

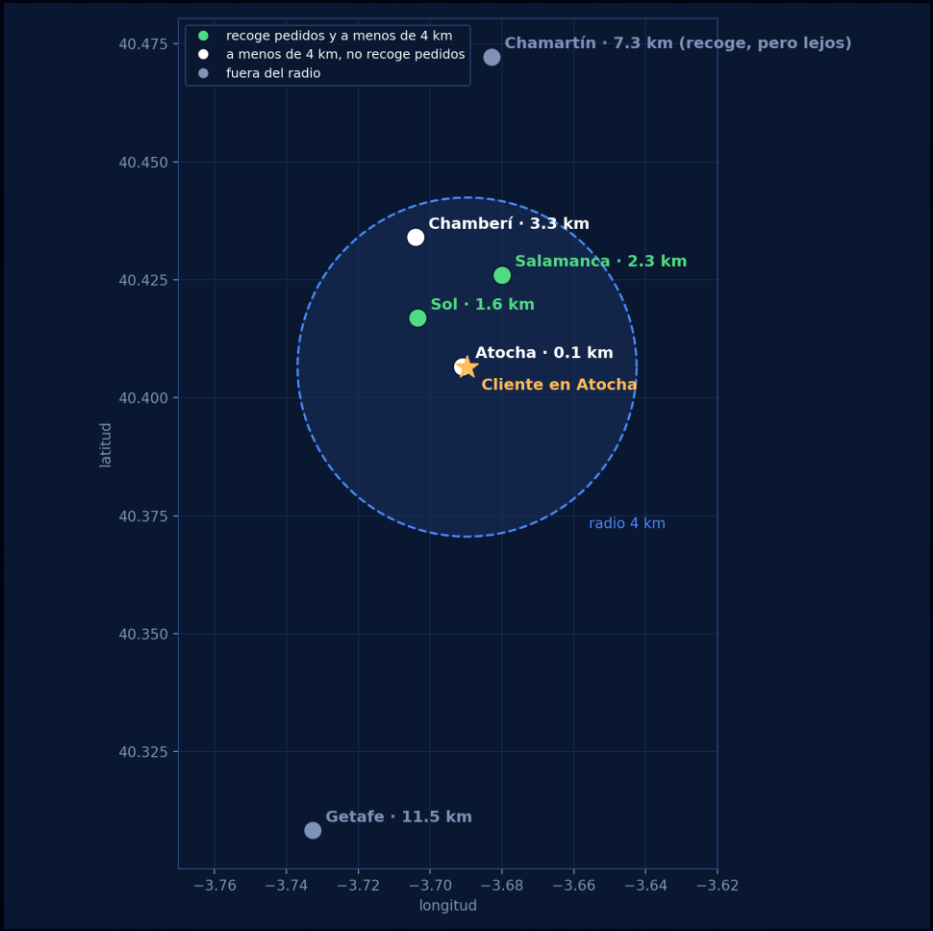


FÍJATE EN

la más cercana, Atocha, no recoge pedidos. Con distancia + significado salen Salamanca y Sol, que sí.

thePower Tech School

Bonus · El mapa



POSTGIS -> MAPA

Lo que ha pasado, dibujado

- Estrella: el cliente en Atocha; círculo: 4 km de radio
- Verde: recogen pedidos y están dentro del radio
- Blanco: dentro del radio, pero no recogen pedidos
- Chamartín recoge pedidos... pero está a 7,3 km
- Coordenadas leídas de PostGIS, no dibujadas a mano

thePower Tech School

Bonus · Anatomía: vector + punto en la misma fila

05_bonus_gis.py

```python
# 1) Tiendas fisicas: datos normales + un vector (que ofrecen) + un punto en el mapa (donde estan)
conn.execute("DROP TABLE IF EXISTS tiendas_fisicas")
conn.execute(f"""                                                                    (1)
    CREATE TABLE tiendas_fisicas (
        id        SERIAL PRIMARY KEY,
        nombre    TEXT,
        servicios TEXT,
        embedding VECTOR({DIMENSIONES}),
        ubicacion GEOGRAPHY(POINT, 4326)      -- longitud/latitud reales (WGS84, como un GPS)
    )
""")
TIENDAS = [  # (nombre, servicios, latitud, longitud)
    ("Sol", "Recogida de pedidos online y cambios de talla.", 40.4169, -3.7035),
    ("Chamberí", "Taller de arreglos de ropa y cambios de talla.", 40.4340, -3.7040),
    ("Salamanca", "Personal shopper con cita previa y recogida de pedidos.", 40.4260, -3.6800),
    ("Chamartín", "Taquillas para recoger pedidos las 24 horas y punto de devolución.", 40.4722, -3.6829),
    ("Atocha", "Punto de devoluciones y taller de reparación de bicicletas.", 40.4066, -3.6910),
    ("Getafe", "Outlet con cambios de talla y devoluciones.", 40.3083, -3.7327),
]
vectores = embedding([t[1] for t in TIENDAS])
with conn.cursor() as cur:                                                           (2)
    cur.executemany("""INSERT INTO tiendas_fisicas (nombre, servicios, embedding, ubicacion)
                       VALUES (%s, %s, %s, ST_SetSRID(ST_MakePoint(%s, %s), 4326)::geography)""",
                    [(n, s, v, lon, lat) for (n, s, lat, lon), v in zip(TIENDAS, vectores)])
conn.execute("CREATE INDEX ON tiendas_fisicas USING gist (ubicacion)")   # indice espacial   (3)
```

1. **La tabla**
   embedding VECTOR(384) y ubicacion GEOGRAPHY(POINT, 4326)

2. **El punto**
   ST_MakePoint(longitud, latitud): ¡primero la longitud!

3. **Índice GiST**
   el índice espacial, como HNSW pero para el mapa

thePower Tech School

Bonus · Anatomía: la consulta híbrida

05_bonus_gis.py

```python
# 3b) Significado + distancia: a menos de 4 km Y ordenadas por lo que ofrecen
print("\nB) A menos de 4 km y ordenadas por SIGNIFICADO:")
for nombre, servicios, sim, km in conn.execute("""                                    (1)
        SELECT nombre, servicios,
               1 - (embedding <=> %s)                                    AS similitud,
               ST_Distance(ubicacion, ST_MakePoint(%s, %s)::geography) / 1000 AS km
        FROM tiendas_fisicas
        WHERE ST_DWithin(ubicacion, ST_MakePoint(%s, %s)::geography, 4000)   -- radio de 4 km  (2)
        ORDER BY embedding <=> %s                                             (3)
        LIMIT 3""", (q, lon, lat, lon, lat, q)):
```

1. **Similitud y km**
   1 - <=> y ST_Distance / 1000, en el mismo SELECT
2. **ST_DWithin**
   solo las que están a menos de 4.000 metros
3. **ORDER BY <=>**
   y de esas, primero la que mejor responde

thePower Tech School

Bonus · Distancias reales en Neon

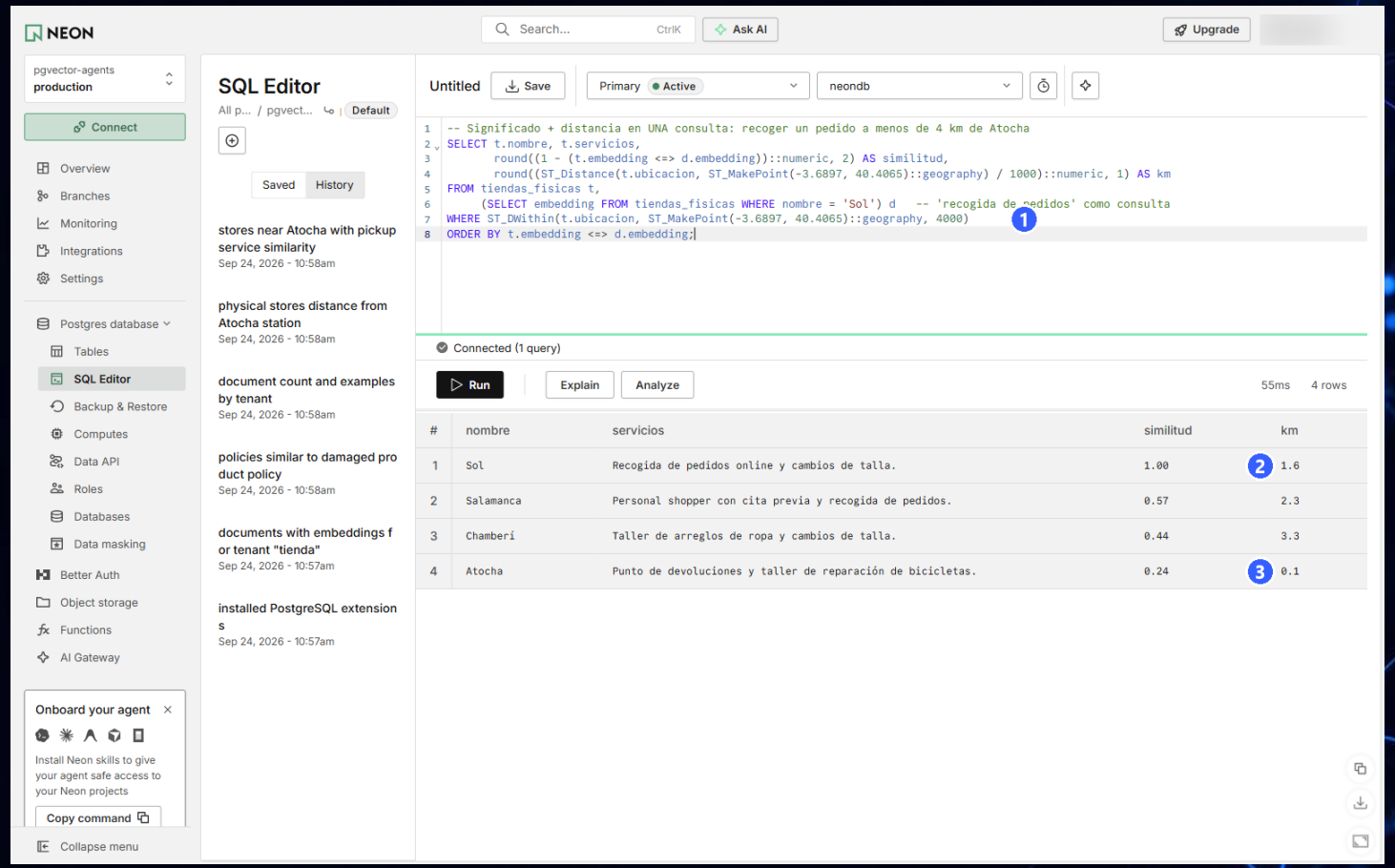
#	nombre	servicios	km	punto
1	Atocha	Punto de devoluciones y taller de reparación de bicicletas.	0.1	POINT(-3.691 40.4066)
2	Sol	Recogida de pedidos online y cambios de talla.	1.6	POINT(-3.7035 40.4169)
3	Salamanca	Personal shopper con cita previa y recogida de pedidos.	2.3	POINT(-3.68 40.426)
4	Chamberí	Taller de arreglos de ropa y cambios de talla.	3.3	POINT(-3.704 40.434)
5	Chamartín	Taquillas para recoger pedidos las 24 horas y punto de devolución.	7.3	POINT(-3.6829 40.4722)
6	Getafe	Outlet con cambios de talla y devoluciones.	11.5	POINT(-3.7327 40.3083)

NEON · SQL EDITOR

PostGIS calcula en metros sobre la Tierra

- ST_Distance: distancia real, no en grados
- Atocha 0,1 km ... Getafe 11,5 km
- ST_AsText(ubicacion): el punto legible, POINT(lon lat)
- ORDER BY ubicacion <-> punto: las más cercanas primero

Bonus · Significado + distancia, en SQL puro



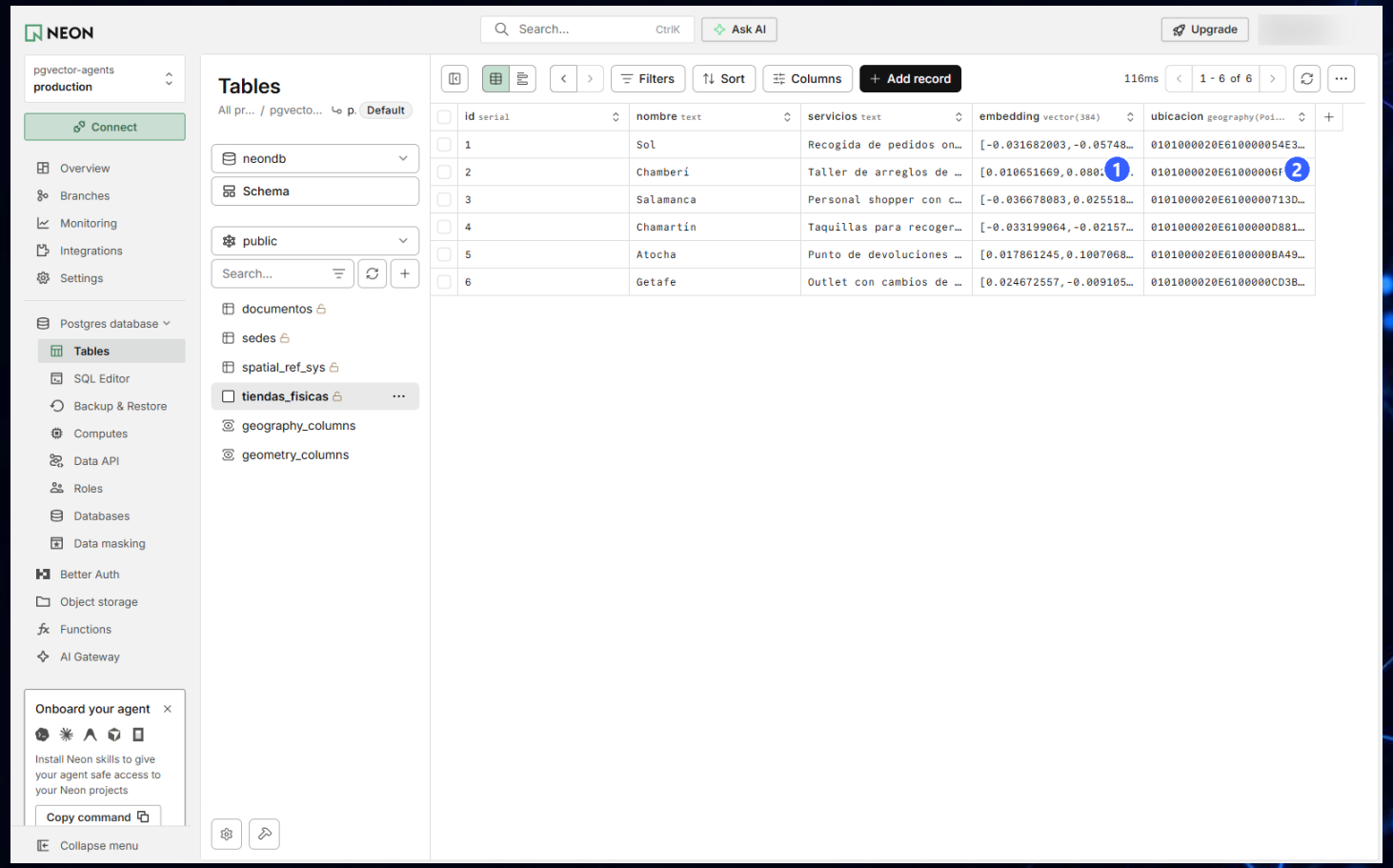
1. **ST_DWithin**
   el radio de 4 km en el WHERE

2. **Sol**
   la usamos de referencia: 'recogida de pedidos' (1,00)

3. **Atocha**
   la más cercana, pero 0,24 de similitud: no es lo que buscas

thePower Tech School

Bonus · La tabla tiendas_fisicas en Neon



1 **embedding**
lo que ofrece cada tienda, en 384 números

2 **ubicacion**
dónde está: PostGIS lo guarda en binario (WKB)

thePower Tech School

Bonus · Pregunta a la clase

PREGUNTA A LA CLASE

¿Qué otros problemas de vuestro trabajo mezclan 'qué' y 'dónde'?

Inmobiliaria, logística, salud, turismo...

thePower Tech School

Bonus · Lo que te llevas

- ✓ **PostGIS** tipo geography, ST_Distance, ST_DWithin e índice GiST
- ✓ **Híbrido** filtrar por distancia y ordenar por significado en una consulta
- ✓ **Todo en Postgres** datos, vectores y mapa en la misma base de datos
- ✓ **Cuidado** ST_MakePoint(lon, lat): la longitud va primero

SIGUIENTE Cierre: el esquema completo, comentado.

thePower Tech School

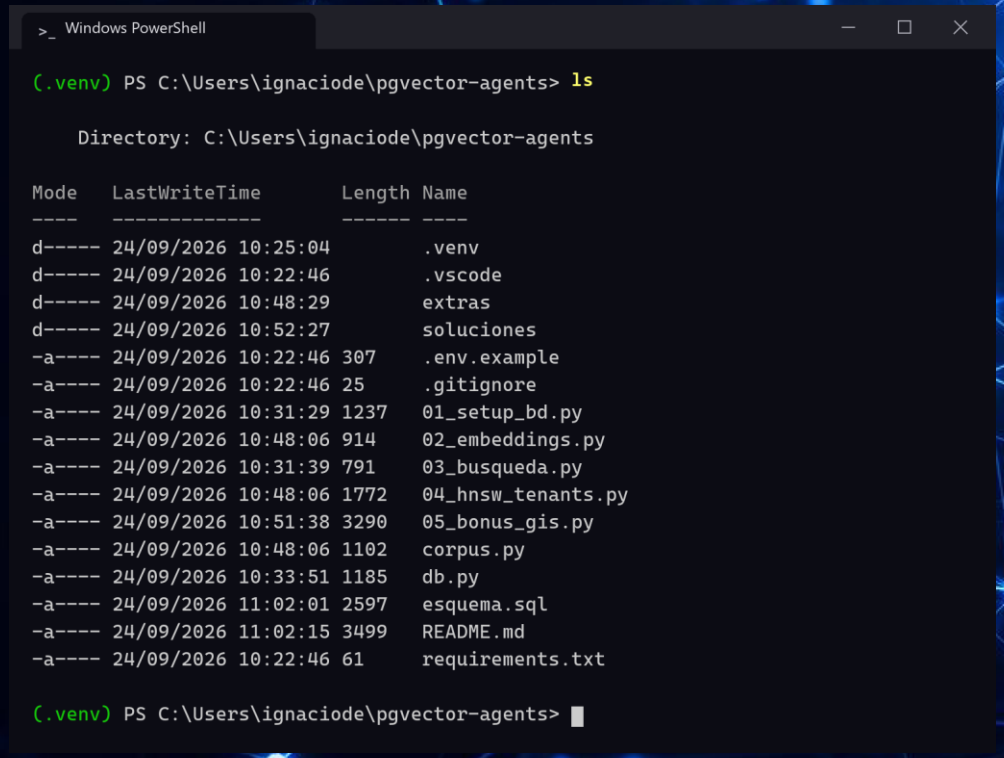
Paso 6 · Cierre y material

QUÉ HACEMOS

- Comentarios didácticos en el código
- esquema.sql comentado
- README con los conceptos
- Proyecto listo para compartir

CONCEPTO CLAVE

El esquema SQL es el mapa del proyecto: extensiones, tabla, índices, aislamiento y bonus GIS en un solo archivo.



FÍJATE EN

un archivo por paso, el esquema SQL, el README, las variaciones en extras y las soluciones.

thePower Tech School

Cierre · El esquema completo, comentado

esquema.sql

```sql
-- 1) Extensiones: Postgres aprende a guardar vectores (y, de bonus, puntos en el mapa)
CREATE EXTENSION IF NOT EXISTS vector;
CREATE EXTENSION IF NOT EXISTS postgis;

-- 2) Documentos: una tabla NORMAL con una columna de 384 numeros
--    (384 = dimensiones del modelo paraphrase-multilingual-MiniLM-L12-v2)
CREATE TABLE documentos (
    id         SERIAL PRIMARY KEY,
    texto      TEXT NOT NULL,                    -- el dato de siempre
    embedding  VECTOR(384),                      -- su significado, en numeros
    tenant_id  TEXT NOT NULL DEFAULT 'tienda'    -- a que cliente pertenece (multi-tenancy)
);

-- 3) Busqueda por significado: los 3 textos mas cercanos a un vector de consulta
--    <=> = distancia coseno (0 = mismo significado). Siempre filtrando por cliente.
-- SELECT texto, 1 - (embedding <=> :consulta) AS similitud
-- FROM documentos
-- WHERE tenant_id = :cliente
-- ORDER BY embedding <=> :consulta
-- LIMIT 3;

-- 4) Indice HNSW: busqueda aproximada muy rapida (a partir de miles de filas se nota)
CREATE INDEX documentos_embedding_hnsw ON documentos USING hnsw (embedding vector_cosine_ops);

-- 5) Aislamiento en la propia base de datos (Row-Level Security)
--    La app se conecta con un rol SIN privilegios; el propietario se salta RLS.
ALTER TABLE documentos ENABLE ROW LEVEL SECURITY;
CREATE POLICY aislamiento_tenant ON documentos
    USING (tenant_id = current_setting('app.tenant_id', true));
-- En cada conexion de la app:  SELECT set_config('app.tenant_id', 'tienda', false);
```

1. **Extensiones**
   vector y postgis
2. **La tabla**
   texto + embedding + tenant_id
3. **HNSW**
   el índice vectorial
4. **RLS**
   la política de aislamiento

thePower Tech School

¿Dónde pongo mi Postgres con pgvector?

	Docker	Neon / Supabase	Postgres local
Instalación	Docker Desktop + una imagen	Nada: cuenta gratuita	Instalador + compilar pgvector (Windows)
pgvector	Incluido (pgvector/pgvector)	Incluido	No siempre viene incluido
PostGIS	Otra imagen (postgis)	Incluido	Stack Builder / paquete aparte
Datos	En tu máquina	En su nube (UE)	En tu máquina
Coste	0 EUR	0 EUR (plan gratuito)	0 EUR
Ideal para	Desarrollo con Docker	Clase, prototipos, portátiles sin Docker	Entornos sin internet

El código es el mismo en los tres: solo cambia DATABASE_URL en el .env.

thePower Tech School

Recap: de la teoría al código

Concepto de la teoría	Dónde lo hemos visto	La pieza clave
Embeddings y similitud	pasos 2 y 3	`384 números; 1 - (embedding <=> pregunta)`
Postgres + pgvector	paso 1	`CREATE EXTENSION vector · VECTOR(384)`
Búsqueda exacta vs. HNSW	paso 4	`USING hnsw · EXPLAIN: Seq Scan / Index Scan`
Operador de vecinos	paso 3	`ORDER BY embedding <=> q LIMIT 3`
Multi-tenancy	paso 4	`WHERE tenant_id = ... · Row-Level Security`
Bonus: GIS	bonus	`GEOGRAPHY(POINT) · ST_DWithin · ST_Distance`

Lo que falta para un RAG: pasarle esos documentos al modelo para que responda con ellos. Eso es la sesión 10.

thePower Tech School

Ejercicio 1 · Búsqueda por significado con pgvector

QUÉ CONSTRUIR

BÁSICO-MEDIO · 1,5 h

Guardar 5 frases con su embedding y, dada una consulta, devolver las 2 más parecidas por significado.

PASOS SUGERIDOS

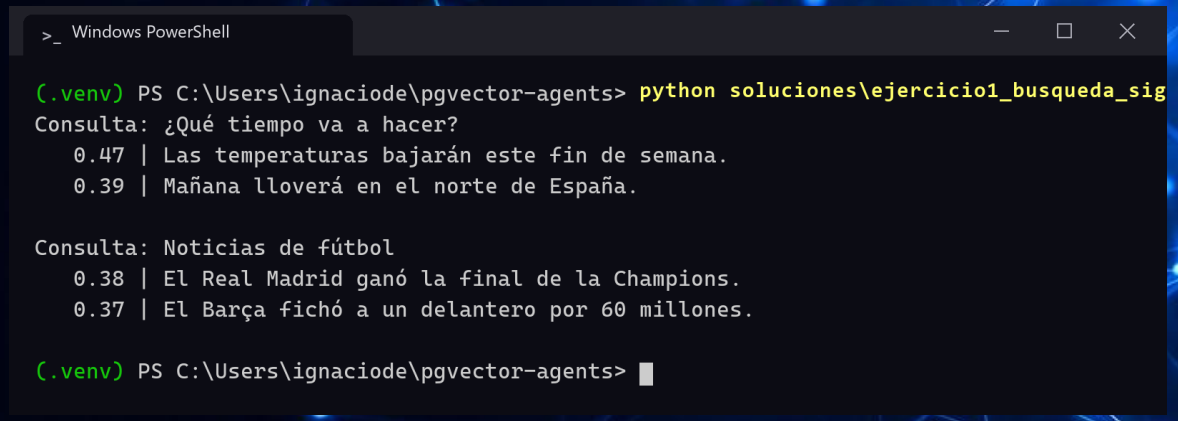
1. Postgres + pgvector (Docker, Neon o Supabase)
2. Generar el embedding de 5 frases
3. Guardarlas en una tabla
4. Consultar y traer las 2 más cercanas

ENTREGABLE

Script + salida con la consulta y las 2 frases más similares.

PARA NOTA · GIS

Da a cada frase una ciudad (punto PostGIS) y busca solo en un radio.



ASÍ DEBE VERSE

Ejecución real de la solución (soluciones/): el tiempo con el tiempo y el fútbol con el fútbol.

thePower Tech School

Ejercicio 2 · Reto: RAG multi-tenant con inteligencia geoespacial

QUÉ CONSTRUIR

RETO

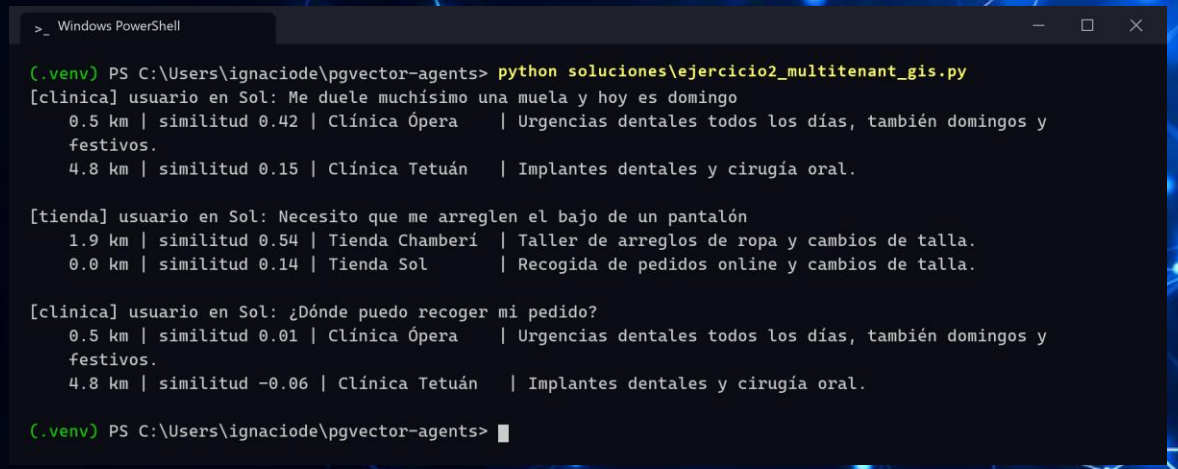
1. **Aislamiento por cliente**
   tenant_id (o RLS): cada organización solo ve lo suyo

2. **Búsqueda por significado**
   embeddings de un corpus propio y <=>

3. **Geolocalización (GIS)**
   sedes con su punto PostGIS y radio con ST_DWithin

ENTREGABLE

Repo con Postgres + pgvector (+ PostGIS), el corpus con embeddings y una demo de que un cliente nunca ve los datos del otro.



id serial	tenant_id text	nombre text	servicios text	embedding vector(384)	ubicacic
1	tienda	Tienda Sol	Recogida de pedidos on…	[-0.031682003,-0.05748…	0101006
2	tienda	Tienda Chamberí	Taller de arreglos de …	[0.010651669,0.0802109…	0101006
3	tienda	Tienda Chamartín	Taquillas para recoger…	[-0.0010388441,-0.0153…	0101006
4	clinica	Clínica Ópera	Urgencias dentales tod…	[-0.046816632,0.012360…	0101006
5	clinica	Clínica Retiro	Ortodoncia invisible y…	[-0.08879517,0.0081903…	0101006
6	clinica	Clínica Tetuán	Implantes dentales y c…	[-0.09104977,0.0141370…	0101006

thePower Tech School